

Open Prompting Guide

A Practical Guide to Getting Better Results from Large Language Models

Khaled Lajili

Version 1.0

github.com/khaledlajili/open-prompting-guide

Licensed under CC-BY-SA

Contents

1 Introduction to Prompt Engineering **5**
- 1.1 What Is a Prompt . . . 5
- 1.2 What Is Prompt Engineering . . . 5
- 1.3 Why Prompting Matters: How Language Models Really Work . . . 6
- 1.4 Prompt Elements . . . 6
- 1.5 Tips for Crafting Clear and Effective Prompts . . . 7
- 1.6 Try It Yourself . . . 7
- 1.7 What's Next . . . 9

2 Zero-Shot & Few-Shot Prompting **10**
- 2.1 What Is Zero-Shot Prompting . . . 10
- 2.2 What Is Few-Shot Prompting . . . 11
- 2.3 Try It Yourself . . . 13
- 2.4 What's Next . . . 14

3 Chain-of-Thought Prompting **15**
- 3.1 What Is Chain-of-Thought Prompting . . . 15
- 3.2 CoT Prompting Techniques . . . 16
- 3.3 When Chain-of-Thought Works Best . . . 19
- 3.4 Try It Yourself . . . 20
- 3.5 What's Next . . . 22

4 Meta Prompting **23**
- 4.1 What Is Meta Prompting . . . 23
- 4.2 How Meta Prompting Works . . . 23
- 4.3 Meta Prompting vs. Few-Shot Prompting . . . 25
- 4.4 Advantages and Limitations of Meta Prompting . . . 26
- 4.5 Try It Yourself . . . 26
- 4.6 What's Next . . . 27

5 Self-Consistency **28**
- 5.1 What Is Self-Consistency? . . . 28
- 5.2 How Self-Consistency Works . . . 28
- 5.3 When to Use Self-Consistency . . . 29
- 5.4 Try It Yourself . . . 29
- 5.5 What's Next . . . 30

6 Generated Knowledge Prompting **31**
- 6.1 What Is Generated Knowledge Prompting . . . 31
- 6.2 How Generated Knowledge Prompting Works . . . 31
- 6.3 When to Use Generated Knowledge Prompting . . . 32
- 6.4 Try It Yourself . . . 33
- 6.5 What's Next . . . 34

7 Prompt Chaining **35**
- 7.1 What Is Prompt Chaining? . . . 35
- 7.2 Why Use Prompt Chaining? . . . 35

7.3 How Prompt Chaining Works . . . 36
7.4 Common Use Cases for Prompt Chaining . . . 38
7.5 Try It Yourself . . . 39
7.6 What's Next . . . 40
7.7 What's Next . . . 40

8 Tree of Thoughts (ToT) 42
8.1 What Is Tree of Thoughts? . . . 42
8.2 How Tree of Thoughts Works . . . 42
8.3 Search Strategies for Tree of Thoughts . . . 43
8.4 When Tree of Thoughts Works Best . . . 44
8.5 Try It Yourself . . . 45
8.6 What's Next . . . 46

9 Retrieval-Augmented Generation (RAG) 47
9.1 What Is Retrieval-Augmented Generation? . . . 47
9.2 How the RAG Pipeline Works . . . 47
9.3 RAG vs. Fine-Tuning: When to Use Which? . . . 48
9.4 RAG vs. Generated Knowledge Prompting . . . 49
9.5 When to Use RAG . . . 49
9.6 Try It Yourself . . . 49
9.7 What's Next . . . 50

10 Automatic Reasoning and Tool-use (ART) 52
10.1 What Is Automatic Reasoning and Tool-use (ART)? . . . 52
10.2 How ART Works . . . 52
10.3 Key Characteristics and Advantages . . . 52
10.4 Try It Yourself . . . 53
10.5 What's Next . . . 54

11 Automatic Prompt Engineer (APE) 55
11.1 What Is APE? . . . 55
11.2 How APE Works . . . 55
11.3 Advantages and Limitations of APE . . . 56
11.4 Try It Yourself . . . 57
11.5 What's Next . . . 58

12 Active-Prompt 59
12.1 What Is Active-Prompt? . . . 59
12.2 How Active-Prompt Works . . . 59
12.3 A Simple Example . . . 60
12.4 When to Use Active-Prompt . . . 60
12.5 Try It Yourself . . . 61
12.6 What's Next . . . 62

13 Directional Stimulus Prompting (DSP) 63
13.1 What Is Directional Stimulus Prompting? . . . 63
13.2 How Directional Stimulus Prompting Works . . . 63
13.3 Directional Stimulus Prompting vs. Standard Prompting . . . 65

13.4 When to Use Directional Stimulus Prompting 65
13.5 Try It Yourself 65
13.6 What's Next 66

14 Program-Aided Language Models (PAL) 67
14.1 What Is Program-Aided Language Models (PAL)? 67
14.2 How Program-Aided Language Models Works 67
14.3 Advantages and Limitations of PAL 68
14.4 Try It Yourself 69
14.5 What's Next 69

15 ReAct Prompting 70
15.1 What Is ReAct? 70
15.2 How ReAct Works 70
15.3 Try It Yourself 71
15.4 What's Next 72

16 Reflexion 73
16.1 What Is Reflexion? 73
16.2 How Reflexion Works 73
16.3 When to Use Reflexion 74
16.4 Try It Yourself 74
16.5 What's Next 74

17 Multimodal Chain-of-Thought Prompting 76
17.1 What Is Multimodal CoT 76
17.2 How Multimodal CoT Works 76
17.3 Try It Yourself 77
17.4 What's Next 78

18 Introduction to AI Agents 79
18.1 What Is an AI Agent? 79
18.2 Core Components of an AI Agent 79
18.3 Why Build with Agents 80
18.4 Common Use Cases for AI Agents 80
18.5 Try It Yourself 81
18.6 What's Next 81

19 AI Workflows vs. AI Agents 82
19.1 What Are Agentic Systems 82
19.2 AI Workflows 82
19.3 AI Agents 83
19.4 Key Differences 83
19.5 Choosing the Right Approach 83
19.6 Try It Yourself 84
19.7 What's Next 84

20 Context Engineering for AI Agents 85
20.1 What Is Context Engineering 85

20.2 Why Context Engineering Matters 85
20.3 Common Problems and Fixes 85
20.4 Practical Principles 86
20.5 Try It Yourself 86
20.6 What's Next 87

21 Context Engineering Deep Dive: Building a Deep Research Agent 88
21.1 Why Context Engineering Requires Iteration 88
21.2 Designing the Research Agent's Architecture 89
21.3 Engineering the Main Agent's System Prompt 91
21.4 Defining Tools Clearly 93
21.5 Choosing Between Flexible and Strict Instructions 95
21.6 Improving Research Task Planning 97
21.7 Designing Communication Between Agents 99
21.8 Managing Context Length 101
21.9 Adding Error Handling 103
21.10 The Context Engineering Improvement Cycle 104
21.11 Try It Yourself 105
21.12 What's Next 107

22 Function Calling in AI Agents 108
22.1 What Is Function Calling? 108
22.2 How Function Calling Powers AI Agents 109
22.3 The Role of Tool Definitions 112
22.4 Understanding the Agent Loop: Actions and Observations 115
22.5 Debugging Function Calls 117
22.6 Best Practices for Function Calling 120
22.7 Try It Yourself 123
22.8 What's Next 125

23 Deep Agents 126
23.1 What Are Deep Agents? 126
23.2 Planning: Turning Large Goals into Manageable Tasks 127
23.3 Orchestrator and Sub-Agent Architecture 129
23.4 Context Retrieval and Persistent Memory 131
23.5 Context Engineering for Deep Agents 133
23.6 Verification: Checking the Agent's Work 135
23.7 Putting the Components Together 137
23.8 When Should You Build a Deep Agent? 139
23.9 Try It Yourself 140
23.10 What's Next 141

1 Introduction to Prompt Engineering

You've probably chatted with an AI assistant — maybe asked it to write an email, explain a topic, or fix a bit of code. Sometimes it gives exactly what you wanted, and sometimes it completely misses the point.

The difference isn't magic — it's **how you ask**.

This guide will show you how to turn your words into clear instructions that AI systems can actually understand — helping you get more accurate, creative, and useful results.

Every AI assistant you've used — like ChatGPT or Claude — is powered by a **Large Language Model (LLM)**, a system that generates text based on your instructions.

1.1 What Is a Prompt

A **prompt** is the text instruction you give to a language model to get a response. It can be a question, a task description, a block of structured text, or even an example to follow.

Simple examples:

```
- "Translate this sentence into Spanish."
- "Write a friendly message to reschedule a meeting."
- "List three fun facts about dolphins."
```

Each one tells the model:

1. **What you want it to do** (e.g., translate, write, list)
2. **What information or context to use** (e.g., "this sentence", "reschedule a meeting", "dolphins")
3. **How the response should be framed** (e.g., in Spanish, friendly tone, as a list)

1.2 What Is Prompt Engineering

Prompt engineering means **designing and refining prompts** so that an LLM reliably delivers accurate, well-organized, and useful responses.

It's a mix of:

- **Communication skills** — writing clear, specific instructions the model can understand
- **System thinking** — structuring tasks, examples, and context in a logical way
- **Experimentation** — iterating, testing, and improving results

Real-world examples:

- Marketers create prompts that generate consistent brand voice.
- Developers write structured prompts to produce code safely.
- Students use reasoning prompts to explain complex concepts.

1.3 Why Prompting Matters: How Language Models Really Work

Language models can't read your mind — they don't "understand" your goals or intentions. Instead, they:

1. Predict one word at a time based on patterns in the input
2. Rely completely on the prompt and context you give them

That's why **prompt engineering** is important: It helps you turn your intention into the kind of input a model can actually work with.

> ℹ **Think of the model like a smart autocomplete :** It doesn't know what you want — only what's likely to come next based on what you've typed.

1.4 Prompt Elements

A clear prompt often contains a few simple parts.

Element	What It Means	Example
Instruction	What you want the model to do.	"Summarize this article."
Input / Context	The information you want the model to use.	"The article below explains our company goals."
Output Format	How you want the answer to be structured.	"Respond in 3 bullet points."
Constraints	Extra rules like tone, style, or length.	"Use formal tone. Max 100 words."
Role / Persona	Who the model should pretend to be.	"You are a professional copywriter."

> ℹ **Use what you need :** You don't need all of them every time, but knowing them helps you build better prompts.

1.5 Tips for Crafting Clear and Effective Prompts

Good prompts are **specific**, **context-aware**, **constrained**, and **structured**.

#	Tip & Explanation	Example
1	**Start simple:** Describe what you actually want the model to do — one clear action.	"Write a report on renewable energy."
2	**Be specific:** Be explicit about the topic, scope, and focus.	"Write a report comparing solar and wind energy."
3	**Add context:** Give background, purpose, or audience.	"Write a report comparing solar and wind energy for a high-school science project."
4	**Define output format:** Specify how you want the answer delivered.	"Write a short report comparing solar and wind energy in three paragraphs: introduction, comparison, and conclusion."
5	**Set constraints:** Limit or guide the response (e.g., length, tone, focus, style) so it stays on target.	"Write a 200-word formal report comparing solar and wind energy, focusing on advantages and disadvantages only."
6	**Use positive, precise phrasing:** Tell the model what to do, not what not to do; avoid vague negatives.	"Don't make it too long." ▯ "Keep it under 200 words."
7	**Assign a role or perspective:** You can define a role to influence style and reasoning, but use it only when it adds value.	"You are a science teacher. Write a report comparing solar and wind energy."
8	**Iterate and refine:** Review the output and adjust your prompt — add missing details or clarify unclear parts.	"You are a science teacher. Write a 200-word formal report comparing solar and wind energy for a high-school science project. Focus on advantages and disadvantages only. Structure your answer in three short paragraphs: introduction, comparison, and conclusion."

💡 **Think of a prompt as a mini-contract :** "Here's what you should do, what you know, the limits you must respect, and how to deliver it."

1.6 Try It Yourself

Let's practice rewriting a vague prompt into strong, specific ones.

Take this vague prompt:

```
Write about healthy food.
```

Your goal is to **turn it into two clear prompts** — one for a creative writing task and one for a factual information task.

When you rewrite, remember to:

- Add **context** (like purpose, or audience).
- Add **constraints** (like length, tone, or focus).
- Specify the **output format** (paragraphs, list, table, etc.).
- Keep each prompt focused on one purpose only.

1.6.1 Example Answers

Creative version

```
Write a short, 150-word persuasive blog post encouraging
teenagers to eat healthier meals. Use an inspiring and friendly
tone.
```

- **context:** persuade, teenagers
- **constraints:** 150 words, inspiring and friendly tone
- **output format:** blog post

Factual version

```
For a high-school health class, write a concise, 120-word
explanation listing 5 scientifically proven health benefits of
a balanced diet. Use clear, neutral language and format your
answer as bullet points.
```

- **context:** high-school health class
- **constraints:** 120 words, 5 benefits, neutral tone
- **output format:** bullet points

Together, they show how changing context and constraints completely changes the model's output — even when the topic is the same.

1.7 What's Next

You now know **how to talk clearly to a language model** — the foundation of everything that follows.

But sometimes, even a perfectly worded prompt fails because the model hasn't seen enough examples of what you mean. That's where **Zero-Shot and Few-Shot prompting** come in — the next step in teaching models through examples.

2 Zero-Shot & Few-Shot Prompting

You've learned how to write clear, structured prompts — but sometimes, even clear instructions lead to confusing results.

Imagine you ask the model:

```
Write a short message to apologize to a friend for missing
their birthday dinner.
Make it sound sincere, friendly, and positive so the friend
forgives me.
```

Sounds clear, right?

Yet the model might write:

> Hey, sorry I couldn't make it to your dinner. Hope it went great!

which is polite but sounds too casual or even indifferent.

> I deeply regret missing your dinner and hope you can find it in your heart to forgive me.

which sounds overly formal and dramatic for a friend.

Both are technically correct, but neither matches the warm, friendly tone you wanted — something like:

> Can't believe I missed your big night, Mia! I'm so sorry — I'll make it up to you this weekend. Hope you had the best time.

The model did what you asked — just not how you meant it. It didn't know your style, because you never showed it what "good" looks like.

Sometimes the task is simple enough that you don't need any examples. That's called **zero-shot prompting**. But other times, showing one or two examples makes all the difference and that's where **few-shot prompting** comes in.

2.1 What Is Zero-Shot Prompting

Zero-shot prompting means giving the model a clear instruction **without any examples or demonstrations**.

The model relies entirely on what it already knows from training to produce the result — like asking:

```
Translate the following English sentence into French: "The
weather is nice today."
```

The model doesn't need you to show examples of translations — it's seen millions of them during training.

2.1.1 When Zero-Shot Works Well

Zero-shot prompting is effective when:

- The task is **common and well-defined** (like translation, summarization, factual Q&A)
- The **desired output format** is obvious from the instruction
- You just need a **quick first attempt** to see how the model interprets your goal

It's perfect for testing ideas or building a baseline before refining your prompt further.

2.1.2 When It Starts to Break Down

Zero-shot prompting struggles when:

- The task is **ambiguous** or **has multiple valid answers**
- The model must follow **specific tone, structure, or style rules** that aren't spelled out
- The output needs to match **your custom definition or criteria**

In these cases, the model can only guess your intent — and its guess might be wrong.

> 💡 **Zero-shot is your starting point :** It teaches you how far clarity alone can take you. When instructions alone can't capture nuance, tone, or structure — that's your signal to use examples.

2.2 What Is Few-Shot Prompting

Few-shot prompting means showing the model **a few examples of how a task should be done** before asking it to handle a new case — This helps it learn your tone, format, or reasoning pattern from those examples.

Let's say you want the model to turn customer feedback into a short, friendly summary.

Without examples:

```
Summarize this review in a friendly tone.
"The new update fixed all the bugs, runs perfectly now."
```

You might get inconsistent results — some formal, some casual.

With examples:

```
Review: "The app is easy to use but drains the battery fast."
Summary: "Great usability, but needs battery optimization."

Review: "Loved the design, but wish it had dark mode."
Summary: "Stylish and smooth — dark mode would make it
perfect!"

Now summarize this one:
Review: "The new update fixed all the bugs, runs perfectly
now."
```

The model learns the rhythm and tone from your examples and produces something like:

> Smooth, stable, and finally bug-free — great job!

It's consistent, friendly, and aligned with your examples — no guessing required.

2.2.1 When to Use Few-Shot Prompting

Few-shot prompting is especially helpful when you want the model to:

- Follow a **specific tone, style, or voice**
- Produce outputs with a **consistent format or structure**
- Learn a **custom rule, scoring system, or labeling style**
- Handle **specialized or unfamiliar tasks** by imitation
- Improve **accuracy and reliability** through clearer context
- Generate **more predictable or repeatable results** across prompts

💡 **Prefer quality over quantity :** Usually, **two to five examples** are enough. More examples can help with consistency, but too many may crowd the context and confuse the model.
The key is **clear, diverse, and consistent demonstrations**.

2.3 Try It Yourself

Let's practice converting a zero-shot prompt into a few-shot prompt.

Task: Decide whether each message is formal or informal, and rewrite the informal ones to sound formal.

1. "Hey, can u send me that file real quick?"
2. "Could you please forward me the file when you have a moment?"
3. "Send me the file now!"

Start with a **zero-shot** prompt, then improve it into a **few-shot** version by adding **two labeled examples**.

2.3.1 Example Answers

Zero-Shot Prompt:

```
Classify each message as formal or informal. If informal,
rewrite it to sound formal.

1. "Hey, can u send me that file real quick?"
2. "Could you please forward me the file when you have a
moment?"
3. "Send me the file now!"
```

Few-Shot Prompt:

```
Classify each message as formal or informal. If informal,
rewrite it to sound formal.

Example 1:
Message: "What's up? Can you send me that report?"
Classification: Informal
Formal Rewrite: "Hello, could you please send me the report?"

Example 2:
Message: "Please find the attached document."
Classification: Formal

Now classify and rewrite:
1. "Hey, can u send me that file real quick?"
2. "Could you please forward me the file when you have a
moment?"
3. "Send me the file now!"
```

Notice how the examples show the model:

- **The exact format** to follow (Message, Classification, Formal Rewrite)
- **What counts as formal vs informal**
- **The style and tone** for rewrites

2.4 What's Next

You've now learned how to **teach a model by example**.

But when your task needs **step-by-step thinking**, **traceable logic**, or **verifiable answers**, it's time to level up to **Chain-of-Thought (CoT)** — the next technique for improving reasoning and accuracy.

3 Chain-of-Thought Prompting

You've seen how examples can shape what a model says. But what if your task is about thinking?

Imagine you ask the model:

```
A train leaves City A at 8 AM traveling 60 km/h.
Another leaves City B at 9 AM traveling 80 km/h toward City A.
The cities are 300 km apart. When do they meet? Give the result
directly.
```

The model might instantly answer:

> They meet at 11:30 AM.

Confident — but wrong. It skipped the math.

If instead you prompt:

```
A train leaves City A at 8 AM traveling 60 km/h.
Another leaves City B at 9 AM traveling 80 km/h toward City A.
The cities are 300 km apart. When do they meet? Let's think
step by step.
```

The model starts to reason:

> 1. The first train travels 60 km in the first hour.
> 2. At 9 AM, 240 km remain between them.
> 3. Their combined speed is 140 km/h □ 240 / 140 ≈ 1.71 hours.
> 4. 1 h 43 min after 9 AM □ 10:43 AM.

This time, it's correct — because you asked it to think out loud.

Sometimes, clarity and examples aren't enough — you need the model to show its reasoning before answering, turning opaque guesses into transparent, verifiable steps.

That's where **Chain-of-Thought** comes in.

3.1 What Is Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting means asking a model to break down its reasoning into **intermediate steps** before giving a final answer.

Instead of jumping straight to a conclusion, the model:

1. Shows each step of its thinking
2. Explains how one step leads to the next
3. Arrives at an answer through visible logic

Think of it like showing your work in math class — the teacher wants to see how you got the answer, not just the answer itself.

Why does this matter?

Language models predict what comes next based on patterns, but they don't actually "understand" logic the way humans do. When you ask them to think step-by-step, you force them to follow a more structured reasoning process — which often leads to more accurate answers, transparent reasoning you can verify, and logic you can check at each step.

> 💡 **Key insight :** CoT prompting doesn't make the model smarter — it changes how the model generates its answer, encouraging a more methodical approach that reduces errors.

3.2 CoT Prompting Techniques

There are three main ways to implement Chain-of-Thought prompting, each with different levels of control and guidance.

3.2.1 Zero-Shot CoT: "Let's think step by step"

Zero-shot CoT means asking the model to reason step-by-step **without giving it any examples**.

All you do is add a simple phrase like "Let's think step by step," "Break this down into steps," or "Explain your reasoning."

Example:

```
Question: If a bakery makes 48 cupcakes and packages them in
boxes of 6,
how many boxes can they fill?
Let's think step by step.
```

Model's response:

> Step 1: We have 48 cupcakes total.
> Step 2: Each box holds 6 cupcakes.
> Step 3: To find the number of boxes, divide: 48 ÷ 6 = 8.
> **Answer:** They can fill 8 boxes.

The model decides how to break down the reasoning on its own.

3.2.2 Few-Shot CoT: Teaching by Example

Few-shot CoT means showing the model **examples of step-by-step reasoning** before asking it to solve a new problem.

You provide two to five demonstrations that include both the reasoning steps and the final answer.

Example:

```
Q: A store has 15 apples. They sell 7 and buy 12 more. How many
apples do they have?
A: Start with 15 apples. Sell 7: 15 - 7 = 8. Buy 12 more: 8 +
12 = 20.
Answer: 20 apples.

Q: A library has 32 books. They donate 8 and receive 15 more.
How many books do they have?
A: Start with 32 books. Donate 8: 32 - 8 = 24. Receive 15 more:
24 + 15 = 39.
Answer: 39 books.

Now solve this:
Q: A classroom has 20 pencils. Students use 9 and the teacher
adds 6 more.
How many pencils are there?
```

Model's response:

> A: Start with 20 pencils. Students use 9: 20 - 9 = 11. Teacher adds 6: 11 + 6 = 17.
> **Answer:** 17 pencils.

The model learns your reasoning style and structure from the examples and applies it to the new problem.

Tips for good Few-Shot CoT examples:

- Use two to five diverse examples (not too similar to each other)
- Show clear, logical steps in each example
- Keep the reasoning concise but complete
- Make sure your examples actually lead to the correct answer

3.2.3 Guided CoT: Providing a Reasoning Framework

Guided CoT means explicitly telling the model **what steps to follow** when reasoning through a problem.

You provide an outline or template of the reasoning process:

Example:

```
Question: A rectangular garden is 12 meters long and 8 meters
wide. What is its area?

Use this reasoning structure:
1. List what we know
2. Determine the formula needed
3. Perform the calculation
4. State the final answer
```

Model's response:

> 1. What we know: Length = 12 meters, Width = 8 meters
> 2. Formula needed: Area of rectangle = Length × Width
> 3. Calculation: 12 × 8 = 96
> 4. **Final answer:** The garden's area is 96 square meters.

This technique gives you **maximum control** over how the model structures its reasoning — useful when you need a consistent approach across many similar problems or want to enforce a specific methodology (like a scientific method, troubleshooting checklist, or decision framework).

3.2.4 When to Use Each Technique

Technique	Best For
Zero-Shot CoT	Quick reasoning tasks, initial testing, straightforward problems
Few-Shot CoT	Tasks needing specific style or format, consistent outputs, complex reasoning
Guided CoT	Standardized processes, compliance needs, enforcing methodology

Start simple, add control as needed : Start with **Zero-Shot CoT** to see if simple reasoning helps. If you need more control or consistency, move to **Few-Shot CoT**. Use **Guided CoT** when you must enforce a specific reasoning process or need reproducible, auditable steps.

3.3 When Chain-of-Thought Works Best

3.3.1 Tasks Where CoT Helps Most

Math Problems Questions that involve arithmetic or formulas. Writing out the steps prevents calculation mistakes.

```
A pair of shoes costs $60. There's a 25% discount. What's the
final price?
```

Word Problems Real-world stories that hide a math or logic question. Step-by-step reasoning helps extract the numbers and operations correctly.

```
Emma has 10 apples, gives away 3, then buys 5 more. How many
apples now?
```

Logic & Deduction Situations that require applying rules carefully. CoT ensures the model checks every condition before concluding.

```
If all cats are mammals and some mammals are pets, are all cats
pets?
```

Multi-Step Decisions Choosing between options using several criteria. Listing pros, cons, and trade-offs makes reasoning transparent.

```
Laptop A is cheaper but heavier. Laptop B is lighter but costs
more.
Which should I buy for travel?
```

Planning Tasks Creating or organizing a sequence of actions. CoT turns vague goals into clear, ordered steps.

```
Plan a weekend trip: choose location → book stay → plan meals →
pack essentials.
```

Cause-and-Effect Reasoning Understanding what happens when something changes. Describing the chain of causes improves logical flow.

```
If it rains tomorrow, what might happen to the outdoor concert?
```

3.3.2 Tasks Where CoT Usually Doesn't Help

Simple factual recall — The model just needs to remember a fact, like "What's the capital of France?"

Creative writing — Step-by-step logic can make stories or poems sound mechanical, like "Write a poem about space exploration."

Definition or lookup tasks — There's no reasoning, just providing a known answer, like "What does 'photosynthesis' mean?"

3.4 Try It Yourself

3.4.1 Task 1: Few-Shot CoT

You want the model to classify customer reviews as Positive or Negative, and explain why in one sentence.

Create a few-shot prompt with two examples, then ask it to classify this new review:

```
The delivery was fast but the product quality disappointed me.
```

Your prompt should teach the model to:

- State the classification clearly
- Explain the reasoning in one sentence
- Consider both positive and negative aspects

3.4.2 Task 2: Guided CoT

You need the model to help calculate how much paint someone needs to buy for their room.

The problem is:

```
My bedroom is 4 meters long and 3 meters wide, with walls that
are 2.5 meters tall.
One paint can covers 10 square meters. How many cans do I need?
```

Create a guided prompt that gives the model a clear step-by-step structure to follow when solving this paint calculation problem. Think about what steps are needed to go from room dimensions to number of paint cans.

3.4.3 Example Answers

Task 1: Few-Shot CoT

```
Review: "Great features but the app crashes frequently."
Classification: Negative
Reasoning: Despite acknowledging good features, the frequent
crashes are a critical
usability issue that outweighs the positives.

Review: "Exactly what I needed, works perfectly!"
Classification: Positive
Reasoning: Completely satisfied with both functionality and
performance, with no
mentioned drawbacks.

Now classify this:
Review: "The delivery was fast but the product quality
disappointed me."
```

Expected model response:

> Classification: Negative
> Reasoning: While fast delivery is positive, the disappointment with product quality is the primary concern and determines overall satisfaction.

Task 2: Guided CoT

```
Problem: "My bedroom is 4 meters long and 3 meters wide, with
walls that are 2.5 meters
tall. One paint can covers 10 square meters. How many cans do I
need?"

Solve this step by step:
1. Calculate the wall area (perimeter × height)
2. Divide by coverage per can
3. Round up to whole cans needed
```

Expected model response:

1. Wall area: Perimeter = 2(4 + 3) = 14 meters. Area = 14 × 2.5 = 35 square meters.
2. Cans needed: 35 ÷ 10 = 3.5 cans.
3. Round up: You need 4 cans (since you can't buy partial cans).

3.5 What's Next

You've now learned how to guide a model through step-by-step reasoning using examples or structured frameworks.

But sometimes, instead of showing the model specific examples or reasoning steps, you can describe the underlying structure of the task itself.

That's where **Meta Prompting** comes in — a technique that focuses on abstract patterns and reusable structures to guide the model's responses.

4 Meta Prompting

You've learned how to write clear instructions, teach with examples, and guide step-by-step reasoning.

Imagine you want the model to solve word problems involving quantities. You could show it examples about apples, books, and pencils. But those examples all follow a similar pattern: start with an amount, apply one or more changes, then calculate what remains.

Instead of teaching the model through several specific examples, you can describe the underlying pattern.

This is the idea behind **Meta Prompting**.

4.1 What Is Meta Prompting

Meta Prompting is a technique that focuses on the **structure and syntax** of tasks rather than their specific content.

Instead of showing the model detailed examples full of concrete facts, you give it an abstract framework that describes the form a good solution should take.

The key characteristics of meta prompting are:

1. **Structure-oriented** — Prioritizes the format and pattern of problems and solutions over specific content.
2. **Syntax-focused** — Uses a fixed template or format that guides how the answer should be built.
3. **Abstract examples** — Shows the skeleton of good solutions without filling in concrete content.
4. **Versatile** — Works across many different domains and problem types.
5. **Categorical approach** — Organizes the prompt by clearly defining the type of task and the logical parts it should contain.

4.2 How Meta Prompting Works

Meta prompting starts by identifying the pattern behind a task, then expressing that pattern in a form the model can reuse.

4.2.1 Step 1: Identify the Task Structure

Ask what the task is made of, regardless of its specific content.

For a quantity-based word problem, the structure might be:

```
Starting quantity → Changes → Target quantity
```

For a customer-feedback summary, it might be:

```
Customer experience → Main concern or benefit → Overall
sentiment → Key takeaway
```

The goal is to find the pattern that remains useful when the details change.

4.2.2 Step 2: Define the Components

Describe the role of each component so the model knows what to identify.

For example:

```
- Starting quantity: the amount at the beginning
- Ordered changes: events that increase or decrease the amount
- Target quantity: the quantity the question asks us to find
```

These definitions guide the model without requiring multiple worked examples.

4.2.3 Step 3: Define the Expected Solution Structure

Specify how the model should organize its response.

```
1. Identify the starting quantity.
2. Translate each change into a mathematical operation.
3. Apply the operations in order.
5. Give the final answer with its unit.
```

4.2.4 Step 4: Apply the Structure to a New Input

Now give the model a new problem and ask it to use the structure.

Example prompt:

```
Solve the following quantity-based word problem using this
abstract structure.

Problem structure:
- Starting quantity
- Ordered changes
- Target quantity

Solution structure:
```

```
1. Identify the starting quantity.
2. Translate each change into a mathematical operation.
3. Apply the operations in order.
5. Give the final answer with its unit.

Problem:
A shop starts the day with 120 notebooks. It sells 35 notebooks
and receives a delivery of 18 more. How many notebooks does it
have now?
```

Model's response:

> 1. Starting quantity: 120 notebooks.
> 2. Ordered changes: Sells 35 (subtract 35); Receives 18 (add 18).
> 3. Calculation: 120 – 35 = 85; 85 + 18 = 103.
> 5. **Answer: 103 notebooks.**

4.3 Meta Prompting vs. Few-Shot Prompting

Meta prompting and few-shot prompting can both help models produce more consistent answers, but they guide the model in different ways.

Aspect	Few-Shot Prompting	Meta Prompting
Main focus	Specific demonstrations	Abstract structure and patterns
What you provide	Examples with expected outputs	A reusable framework or representation
How it guides the model	Shows what a good answer looks like	Describes how the task is organized
Token use	Depends on the number and length of examples	Can be more token-efficient when a compact structure replaces many examples
Best suited for	Custom styles, labels, and patterns that are easiest to demonstrate	Tasks with a recognizable structure that can be reused across different inputs

💡 **Choose the guidance that fits the problem :** Use few-shot prompting when the model needs to imitate a particular style or learn a custom convention from examples. Use meta prompting when you can describe the underlying structure clearly and want to reuse it across different inputs.

4.4 Advantages and Limitations of Meta Prompting

4.4.1 Why Use Meta Prompting?

Advantage	Why It Matters
Token efficiency	You don't need long concrete examples, so the prompt stays short.
Fairer comparison	Models are judged more on their reasoning ability than on how well they copy specific examples.
Zero-shot flavor	Because the influence of particular content is minimized, it behaves like a stronger form of zero-shot prompting.
Better generalization	The model learns a reusable pattern it can apply to unseen problems of the same type.
Clearer debugging	When the output is wrong, you can usually see which part of the structure broke.

⚠ **Structure is guidance, not a guarantee :** Meta prompting still assumes the model already has some knowledge of the domain. On completely novel tasks it can degrade, just like pure zero-shot prompting.

4.4.2 When Can It Be Less Effective?

It may be less effective when:

- The task is purely creative and structure would feel constraining.
- You need the model to match a very specific tone or voice (few-shot examples usually win here).
- The domain is so novel that the model has almost no prior knowledge to draw on.

4.5 Try It Yourself

Task: Create a meta prompt that forces the model to analyze any customer complaint using a consistent diagnostic structure.

Your meta prompt should:

- Define the task category clearly
- Provide a reusable step-by-step structure
- Work for any complaint, not just one specific example

Then test it on this complaint:

```
The package arrived two days late and the product was damaged.
```

4.5.1 Example Answers

Meta Prompt:

```
You are a customer-experience analyst.
For every customer complaint, follow this exact structure:

1. Summarize the core issue in one sentence
2. Identify the customer's primary emotion
3. List the concrete failures (delivery, product,
communication, etc.)
4. Rank the failures by severity (High / Medium / Low)
5. Propose one immediate recovery action and one systemic
improvement

Analyze the following complaint using only the structure above:
The package arrived two days late and the product was damaged.
```

Expected model response:

> 1. The customer received a damaged product two days after the promised delivery date.
> 2. Frustration and disappointment.
> 3. Late delivery; physical damage to the product.
> 4. Product damage = High; Late delivery = Medium.
> 5. Immediate: Offer a full refund or replacement with expedited shipping. Systemic: Review packaging standards and carrier performance for that route.

4.6 What's Next

You've now learned how to guide a model by describing the underlying structure of a task rather than relying on specific examples.

But even when a model follows the right structure, a single reasoning attempt can still lead to an incorrect answer.

That's where **Self-Consistency** comes in — a technique that generates multiple reasoning paths and compares their final answers to help identify more reliable results.

5 Self-Consistency

Chain-of-Thought (CoT) prompting encourages language models to show their work step-by-step. But what happens if the model makes a minor arithmetic or logical misstep early on, the entire final answer falls apart.

Instead of relying on a single chain of thought, what if you generated multiple independent reasoning paths and took the majority vote?

This is the core concept behind **Self-Consistency**.

5.1 What Is Self-Consistency?

Self-Consistency is a prompting technique that generates multiple reasoning paths for the same question, then selects the answer that appears most often.

It is often combined with Chain-of-Thought prompting: the model tries different ways to solve a problem, and their final answers are compared.

5.2 How Self-Consistency Works

5.2.1 Step 1: Prompt with Chain-of-Thought

Give the model a problem using Zero-Shot CoT ("Let's think step by step") or Few-Shot CoT examples showing step-by-step solutions.

5.2.2 Step 2: Generate Multiple Reasoning Paths

Query the model multiple times (e.g., 3 to 10 runs) with a non-zero temperature parameter. This introduces slight variability, allowing the model to explore different ways to break down the problem.

5.2.3 Step 3: Vote on the Final Answer

Aggregate the final answers from all generated paths. The answer that appears most frequently is chosen as the true output.

Example prompt:

```
When I was 6 my sister was half my age. Now I'm 70, how old is
my sister?
```

A single greedy run might over-simplify the math and output 35 (assuming she is always half your age).

With Self-Consistency we sample several paths. Typical outputs look like:

> Path 1: When I was 6 she was 3. Now I am 70, so she is 70 – 3 = 67.

> Path 2: Sister was 3 when I was 6. Age difference is 3 years. 70 – 3 = 67.

> Path 3: Half of 70 is 35, so she is 35 now. incorrect path

5.3 When to Use Self-Consistency

It can be useful for tasks where a small reasoning mistake changes the answer, such as:

- **Complex Arithmetic & Word Problems:** Math tasks where intermediate calculation mistakes are common.
- **Logic & Commonsense Reasoning:** Problems with multiple valid ways to deduce the correct answer.
- **High-Stakes Classification:** Scenarios where accuracy is far more important than token cost or latency.

It is less useful for simple tasks with an obvious answer, and generating multiple attempts takes more time and tokens. If every attempt makes the same mistake, self-consistency will not fix it.

5.4 Try It Yourself

Ask a model to solve this problem in five independent attempts, then compare the final answers:

```
Solve this probability problem:
A fair coin is tossed repeatedly until one of these two
patterns of five consecutive tosses appears:
- Pattern A: HHTHH
- Pattern B: HTHTT
What is the exact probability that Pattern A appears before
Pattern B?
Solve the problem from first principles. Account for
overlapping sequences and express your answer as a simplified
fraction.
Provide your mathematical derivation and final answer.
```

5.4.1 Example Answer

Attempt	Final probability
1	(8/17)
2	(8/17)
3	(8/13)
4	(8/17)
5	(8/17)

the winning answer : 8/17 (appears 4 out of 5 times)

5.5 What's Next

You've now learned how to improve reliability by generating multiple reasoning paths and comparing their answers.

But sometimes, the model needs relevant knowledge before it can reason effectively about a problem.

That's where **Generated Knowledge Prompting** comes in — a technique that asks the model to generate relevant information before using it to answer a question.

6 Generated Knowledge Prompting

You've learned how to guide reasoning step-by-step and sample multiple pathways for accuracy. But what happens when a model fails simply because it makes a wrong factual assumption?

Imagine you ask:

```
Part of golf is trying to get a higher point total than others.
Yes or No?
```

The model might confidently answer:

> Yes.

But that's incorrect. In golf, players generally aim to complete the course using the fewest strokes. The player with the lowest total score wins.

The problem is that the model didn't use the relevant knowledge needed to answer the question correctly.

What if, instead of asking for an answer immediately, you first asked the model to generate relevant knowledge about the topic?

That's the idea behind **Generated Knowledge Prompting**.

6.1 What Is Generated Knowledge Prompting

Generated Knowledge Prompting is a technique that asks a language model to generate relevant information about a topic before using that information to answer.

Instead of answering immediately, the process splits the task into two stages:

1. **Knowledge Generation** — Prompt the model to produce relevant facts, rules, or background information about the topic.
2. **Knowledge Integration** — Combine the original question with the newly generated knowledge to produce the final answer.

6.2 How Generated Knowledge Prompting Works

6.2.1 Step 1: Generate Knowledge

Provide a few examples showing how to generate helpful background context for various statements, followed by your target query.

Prompt:

```
Input: Greece is larger than Mexico.
Knowledge: Greece is approximately 131,957 sq km, while Mexico
is approximately 1,964,375 sq km, making Mexico much larger
than Greece.

Input: Part of golf is trying to get a higher point total than
others.
Knowledge:
```

Model's Generated Knowledge:

> The objective of golf is to complete a set of holes in the fewest number of strokes. Each stroke counts as a point/score, and the player with the lowest total score wins the game.

6.2.2 Step 2: Integrate Knowledge & Answer

Now, feed the generated knowledge back to the model along with the original question.

Prompt:

```
Question: Part of golf is trying to get a higher point total
than others. Yes or No?

Knowledge: The objective of golf is to complete a set of holes
in the fewest number of strokes. Each stroke counts as a
point/score, and the player with the lowest total score wins
the game.

Explain and Answer:
```

Model response:

> No. The objective of golf is not to get a higher point total than others. The goal is to complete the course in the fewest strokes possible, so the player with the lowest score wins.

6.3 When to Use Generated Knowledge Prompting

Generated Knowledge Prompting is most useful when the answer depends on information that the model needs to identify and consider explicitly.

6.3.1 Tasks Where It Can Help

Situation	Why it helps
Commonsense reasoning	Supplies missing real-world facts
Questions that rely on specific knowledge	Reduces reliance on the model's internal (and sometimes flawed) knowledge
Tasks where the model often makes confident but wrong assumptions	Gives it a chance to correct itself before answering

6.3.2 When It May Be Less Effective

Generated Knowledge Prompting may be less useful when:

- The task is simple and the answer is already obvious.
- The model generates incorrect or misleading information.
- The question requires recent or specialized facts that the model does not reliably know.
- The generated information is too broad or unrelated to the question.

In these situations, generating more information can introduce additional errors rather than improve the answer.

> ⚠ **More knowledge does not always mean a better answer :** The model may generate plausible-sounding information that is incorrect. If the final answer depends on that information, the mistake can carry through to the conclusion.

6.4 Try It Yourself

Task: Create a knowledge-generation prompt, then use the generated knowledge to answer this question:

```
A bat is a kind of bird. True or False?
```

6.4.1 Example Answer

Step 1: Generate Knowledge

```
Input: A whale is a fish.
Knowledge: Whales are mammals. They breathe air, give live
birth, and nurse their young.
```

```
Input: A tomato is a vegetable.
Knowledge: Botanically, a tomato is a fruit because it develops
from the flower of the plant and contains seeds.

Input: A bat is a kind of bird.
Knowledge:
```

Possible model response:

> Bats are mammals, not birds. They have fur, give live birth, and produce milk for their young. Birds have feathers and lay eggs.

Step 2 – Final prompt:

```
Question: A bat is a kind of bird. True or False?

Knowledge: Bats are mammals, not birds. They have fur, give
live birth, and produce milk for their young. Birds have
feathers and lay eggs.

Explain and Answer:
```

Expected model response:

> False

6.5 What's Next

You've now learned how to improve a model's answers by generating relevant knowledge before asking it to solve a problem.

The key is to **generate useful information first, then use it to guide the final answer**.

But remember: generated knowledge can be wrong, so important facts should always be checked before you rely on them.

You've now explored several ways to guide a model: clearer instructions, examples, structured reasoning, abstract frameworks, multiple attempts, and additional knowledge.

The next step is to practice combining these techniques and discover which approaches work best for different tasks.

7 Prompt Chaining

You've learned several ways to improve a single prompt: clearer instructions, examples, step-by-step reasoning, multiple attempts, and generated knowledge.

But what happens when a task involves several different operations, and asking the model to perform all of them at once makes the result harder to control?

Imagine you want an AI assistant to answer a question about a long document. The assistant must first find the relevant information, then understand it, and finally formulate an accurate answer.

Instead of asking the model to perform all these operations in a single prompt, you can separate them into smaller steps.

This is the idea behind **Prompt Chaining**.

7.1 What Is Prompt Chaining?

Prompt Chaining is a technique that breaks a complex task into smaller subtasks, where the output of one prompt becomes the input of the next prompt.

Think of it like an assembly line:

```
[ Raw Input ] ⟶ [ Prompt 1: Extract ] ⟶ [ Output 1 ] ⟶ [
Prompt 2: Transform ] ⟶ [ Final Output ]
```

Prompt chaining allows you to control each stage separately, making complex tasks easier to manage, inspect, and improve.

> **The Modular Advantage :** By isolating each subtask into its own prompt, you eliminate the cognitive overload that happens when a model tries to follow dozens of rules simultaneously.

7.2 Why Use Prompt Chaining?

Splitting a complex workflow into multiple steps offers several major advantages over single-prompt approaches:

Advantage	Why It Matters
Higher Accuracy & Reliability	"The model focuses on one simple objective at a time, drastically reducing errors and hallucinations."

Continued on next page

Advantage	Why It Matters
Greater Transparency & Debugging	”If the final answer is wrong, you can inspect each intermediate output to see exactly where the pipeline failed.”
Increased Controllability	”You can clean, reformat, or validate the output of Step 1 before feeding it into Step 2.”
Reusability	”Individual prompts (e.g., a ””Quote Extractor”” or ””Tone Refiner””) can be reused across different workflows.”

7.3 How Prompt Chaining Works

Let’s explore a practical example.

Imagine you are building an AI assistant that answers questions about documents. A user provides a document and asks a question about its content.

Instead of asking the model to read the document and immediately generate an answer, you can create a chain of prompts that handles the task in stages.

7.3.1 Step 1: Extract Relevant Information

The first prompt identifies the passages that are relevant to the user’s question.

Prompt 1:

```
You are an assistant that extracts relevant information from
documents.

Your task is to identify the passages that contain information
needed to answer the given question.

Question:
{{question}}

Document:
####
{{document}}
####

Instructions:
- Extract the most relevant passages from the document.
- Preserve the original wording of the selected passages.
- Do not answer the question yet.
- Do not introduce information that is not present in the
document.
- If no relevant information is found, respond with:
```

```
  "No relevant passages found."

Return only the relevant passages.
```

Suppose the document contains the following information:

> Prompt chaining divides a complex task into smaller sequential subtasks. The output of one prompt becomes the input to the next prompt. This approach can improve controllability and make it easier to identify errors in individual stages.

The user asks:

”What is prompt chaining, and what is one of its benefits?”

The model’s output could be:

> Prompt chaining divides a complex task into smaller
> sequential subtasks.
> The output of one prompt becomes the input to the next prompt.
> This approach can improve controllability and make it easier
> to identify errors in individual stages.

7.3.2 Step 2: Generate the Answer

Now, the second prompt uses the extracted passages to formulate a clear and accurate answer.

Prompt 2:

```
You are a helpful assistant answering questions about
documents.

Use the relevant passages extracted from the document to answer
the original question.

Question:
{{question}}

Relevant passages:
####
{{relevant_passages}}
####

Instructions:
- Answer the question clearly and directly.
- Base your answer on the relevant passages.
```

```
- Do not invent information that the passages do not support.
- If the passages do not contain enough information to answer
  the question, explain that the information is insufficient.

Write the final answer in a concise paragraph.
```

Model response:

> Prompt chaining breaks a complex task into smaller steps, passing each step's output to the next. It improves control over the process and makes errors easier to identify.

7.4 Common Use Cases for Prompt Chaining

Prompt chaining is most useful when a task contains several distinct operations that benefit from being handled separately.

7.4.1 Tasks Where It Can Help

Situation	How Prompt Chaining Helps
Document question answering	Extracts relevant passages before generating an answer.
Research and analysis	Organizes research, analyzes findings, and then prepares a report.
Content creation	Develops an outline, writes a draft, and then improves the writing.
Data processing	Classifies information, extracts specific fields, and transforms the results into a structured format.
AI-powered applications	Separates operations into manageable stages that developers can inspect and improve.

💡 **Keep intermediate outputs structured :** When passing data between chain steps, use clear delimiters like XML tags (<quotes></quotes>), Markdown blocks, or JSON. This helps subsequent prompts easily identify the data they need to process.

7.4.2 When It May Be Less Effective

Prompt chaining may be less useful when:

- The task is simple enough to complete reliably with a single prompt.
- The subtasks do not need to be separated to achieve the desired result.
- The additional steps introduce unnecessary complexity, processing time, or cost.
- An early step generates incorrect information that later steps rely on.

⚠ **More steps do not automatically mean better results :** Errors can propagate through a prompt chain. If the first prompt extracts incorrect information, the second prompt may produce a convincing answer based on that mistake.

7.5 Try It Yourself

Task: Design a 2-step prompt chain to handle a negative customer review.

- **Step 1:** Extract the core issues and customer emotion from the raw review.
- **Step 2:** Use the extracted details from Step 1 to generate an empathetic response email offering a solution.

Raw Review:

```
I ordered my couch three weeks ago and it was supposed to
arrive in five days. When it finally arrived today, the wood on
the left armrest was cracked! Customer support kept me on hold
for 40 minutes when I called.
```

7.5.1 Example Answer

Step 1 — Step 1: Extraction Prompt

```
Extract the key details from the customer review below.

Review:
{{ raw_review }}

Format your output as:
- Primary Emotion: [Emotion]
- Specific Complaints: [Bulleted list of issues]
```

Model output 1:

> - Primary Emotion: Frustrated and disappointed
> - Specific Complaints:
> * Severe shipping delay (3 weeks instead of 5 days)
> * Damaged product (cracked wooden armrest)
> * Poor customer service experience (40-minute hold time)

Step 2 — Response Generation Prompt

```
You are a Customer Experience Manager. Write an empathetic
apology email to a customer based on the following diagnostic
details:

{{ extracted_details }}

Instructions:
- Acknowledge each specific complaint directly.
- Maintain a warm, sincere, and professional tone.
- Offer a replacement armrest/couch and a 20% partial refund
for the inconvenience.
```

Model Output 2:

> Dear Customer,
> I am truly sorry to hear about your experience with your recent couch order. Waiting three weeks for a delivery that was promised in five days is completely unacceptable, and I deeply apologize for the frustration this caused.
> Furthermore, receiving a damaged item with a cracked armrest after that delay — and then having to wait on hold for 40 minutes — is far below the standard of service we aim to provide.
> To make this right, I have personally requested an immediate priority replacement for your couch at no additional cost. Additionally, I have processed a 20% refund back to your original payment method to compensate for the inconvenience and wait time.
> Please let me know if this solution works for you, and I will personally track your replacement shipment.
> Sincerely,
> Customer Experience Team

7.6 What's Next

7.7 What's Next

You've now learned how to break complex tasks into smaller steps, using the output of one prompt as the input for the next.

But what if a problem requires exploring multiple possible approaches before deciding which one is most promising?

That's where **Tree of Thoughts** comes in — a technique that explores multiple reasoning paths, evaluates intermediate ideas, and helps the model navigate toward better solutions.

8 Tree of Thoughts (ToT)

You've seen how Chain-of-Thought guides a model step by step, and how Self-Consistency samples multiple paths and votes on the answer.

But some problems need more than a single linear path or a simple majority vote. They require exploration, evaluation of intermediate ideas, and the ability to backtrack when a direction looks unpromising.

That's the idea behind **Tree of Thoughts (ToT)** — a framework that allows a model to explore multiple intermediate solutions instead of following just one path from beginning to end.

8.1 What Is Tree of Thoughts?

Tree of Thoughts (ToT) is a problem-solving framework that extends Chain-of-Thought prompting by allowing a model to explore, evaluate, and develop multiple candidate solutions.

Instead of generating a single line of reasoning from start to finish, ToT enables the model to:

1. **Branch out:** Generate multiple alternative paths or candidate thoughts at each step.
2. **Self-evaluate:** Assess the progress of each branch (e.g., determining if a path is sure, maybe, or impossible).
3. **Look ahead & Backtrack:** Combine model evaluations with search algorithms (like Breadth-First Search or Depth-First Search) to abandon dead ends and return to promising branches.

> **Beyond Linear Thought :** If Chain-of-Thought is a single path through a maze, Tree of Thoughts is systematically exploring the maze by marking dead ends and retracing steps until you find the exit.

8.2 How Tree of Thoughts Works

8.2.1 Simple ToT-Style Prompt

A practical way to use the idea without building a full search system is the multi-expert prompt:

```
Imagine three different experts are answering this question.
All experts will write down 1 step of their thinking, then
share it with the group.
Then all experts will go on to the next step, etc.
If any expert realises they're wrong at any point then they
leave.
```

```
The question is: [your question]
```

This encourages the model to explore alternative lines of thought and abandon weak ones.

8.2.2 Full Framework

In the complete ToT approach:

- You define how many candidate thoughts to generate at each step
- The model evaluates each candidate
- A search algorithm (BFS, DFS, or beam search) decides which branches to expand
- The process continues until a solution is found or a budget is reached

This version is more powerful but requires additional code and multiple model calls.

8.3 Search Strategies for Tree of Thoughts

8.3.1 Breadth-First Search (BFS)

Breadth-first search explores multiple candidates at the same depth before moving further into their individual branches.

This approach is useful when comparing alternatives at each stage is important.

8.3.2 Depth-First Search (DFS)

Depth-first search follows one candidate through several steps before returning to explore another branch.

If a path fails, the search returns to an earlier decision and tries another route.

8.3.3 Beam Search

Beam search keeps only a limited number of the most promising candidates at each stage.

For example, if the beam width is three, the search retains at most three candidates for further exploration at a given stage.

This reduces the number of branches that need to be explored.

Strategy	How It Explores	Main Trade-off
Breadth-First Search	Explores several candidates at the same depth.	Can use substantial resources when many branches exist.
Depth-First Search	Follows one branch deeply before trying another.	Can spend too much time on an unproductive path.
Beam Search	Keeps a limited number of the most promising candidates.	May discard a path that eventually leads to the best solution.

8.4 When Tree of Thoughts Works Best

Tree of Thoughts is particularly useful for problems where the next step depends on the choices made earlier.

8.4.1 Tasks Where ToT Can Help

Mathematical Puzzles

Problems such as the Game of 24, where different combinations of operations need to be explored.

```
Find an expression that uses the numbers 2, 3, 4, and 6
exactly once to obtain 24.
```

Planning

Tasks where several possible plans must be compared before selecting one.

```
Create three possible approaches for completing a project
within two weeks. Evaluate their risks and feasibility,
then develop the strongest plan.
```

Strategic Decisions

Problems where an early decision affects the possibilities available later.

```
Compare several strategies for launching a new product
with a limited budget. Evaluate the likely trade-offs
and develop the most promising approach.
```

Complex Constraint Problems

Tasks where a solution must satisfy several rules simultaneously.

```
Create a weekly schedule for five employees while respecting
their availability, required working hours, and staffing needs.
Explore different schedules and check each against the
constraints.
```

8.4.2 When ToT May Be Less Effective

Tree of Thoughts is usually overkill for simple questions or tasks that a single Chain-of-Thought already handles well. It also costs more tokens and time because it generates and evaluates many intermediate steps.

8.5 Try It Yourself

Task: Solve the following mathematical puzzle using a Tree of Thoughts approach.

Problem: Use the numbers 2, 3, 4, and 6 exactly once to obtain 24. You may use addition, subtraction, multiplication, division, and parentheses.

8.5.1 Example Answer

```
Solve the Game of 24 using the numbers 2, 3, 4, and 6.

Rules:
- Use each number exactly once.
- You may use +, -, *, /, and parentheses.

Apply a Tree of Thoughts approach:

1. Generate three possible first operations.
2. Evaluate their potential to reach 24.
3. Expand promising candidates into further operations.
4. Backtrack when a candidate cannot lead to a valid solution.
5. Verify the final expression.

Show the main branches briefly and explain why the
successful path works.
```

Expected model response:

> Let's explore three possible first operations:
> 1. **Branch A:** (6 x 4 = 24). The target is reached, but 2 and 3 remain unused. Backtrack.
> 2. **Branch B:** (6 x 3 = 18). Adding the remaining numbers gives (18 + 4 + 2 = 24). Success!
> 3. **Branch C:** (4 x 3 = 12). Multiplying the remaining numbers gives (6 x 2 = 12), so (12 + 12 = 24). Success!
> **Final solutions:** [6 x 3 + 4 + 2 = 24] and [4 x 3 + 6 x 2 = 24].

8.6 What's Next

You've now learned how a model can explore multiple possible solutions, evaluate intermediate ideas, and backtrack when an approach fails.

Unlike a single chain of reasoning, **Tree of Thoughts** helps organize and evaluate alternative paths toward a solution.

But exploring different approaches isn't always enough. Sometimes, a model needs reliable information from external sources before it can produce a good answer.

That's where **Retrieval-Augmented Generation (RAG)** comes in — an approach that retrieves relevant information and provides it as context for generating an answer.

9 Retrieval-Augmented Generation (RAG)

Language models are powerful, but their knowledge is limited to what they learned during training. When a question requires up-to-date, specialized, or highly specific information, the model can guess — and sometimes invent facts.

How can you give a language model direct access to dynamic, external knowledge sources without retraining the entire model?

That's the idea behind **Retrieval-Augmented Generation (RAG)** — an approach that combines information retrieval with language generation to produce answers grounded in external knowledge.

9.1 What Is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is an approach that combines a retrieval system with a language model to generate responses using relevant information from an external knowledge source.

Instead of relying only on what it memorized during training, the model receives additional information retrieved from sources such as documents, databases, or knowledge bases.

A typical RAG system follows three main steps:

1. **Retrieve** — Find information relevant to the user's question from an external knowledge source.
2. **Augment** — Add the retrieved information to the original prompt as additional context.
3. **Generate** — Ask the language model to produce an answer using the provided context.

9.2 How the RAG Pipeline Works

A standard RAG architecture operates in three core phases:

9.2.1 Phase 1: Indexing (Offline Preparation)

Before any questions are asked, your data source needs to be prepared:

1. **Chunking:** Large documents are split into smaller, manageable text segments (e.g., paragraphs or sentences).
2. **Embedding:** An embedding model converts each text chunk into a vector (numerical representation of semantic meaning).
3. **Storage:** The vectors are stored in a vector database (such as FAISS, Pinecone, or Chroma) for fast similarity search.

9.2.2 Phase 2: Retrieval (At Query Time)

When a user asks a question:

1. The system converts the user's query into an embedding vector.
2. It compares the query vector against the vector database.
3. It retrieves the top most relevant text chunks (supporting documents).

9.2.3 Phase 3: Generation (Contextual Fusion)

4. The retrieved text chunks are formatted and concatenated with the original user query inside a structured prompt template.
5. The combined prompt is sent to the LLM.
6. The LLM generates a grounded response based only on the provided context.

9.3 RAG vs. Fine-Tuning: When to Use Which?

Developers often wonder whether they should fine-tune a model or build a RAG pipeline. They serve entirely different purposes:

Feature	Fine-Tuning	Retrieval-Augmented Generation (RAG)
Primary Goal	Teaches the model how to speak (style, tone, format, domain syntax).	Teaches the model facts (supplies external, up-to-date information).
Data Updates	Difficult and expensive (requires retraining model weights).	Instant and cheap (just add/update documents in the database).
Hallucination Risk	Still prone to hallucinations if asked about unseen facts.	Significantly reduced because answers are grounded in retrieved text.
Traceability	Hard to trace where an answer came from.	High traceability (you can cite the exact source document/chunk).

💡 **The Sweet Spot :** In modern AI architectures, you often combine both: fine-tune an open-source model to follow a precise company persona or format, and use RAG to feed it real-time factual data.

9.4 RAG vs. Generated Knowledge Prompting

You've already learned Generated Knowledge Prompting, which also gives a model additional information before it answers.

The two approaches may seem similar, but they obtain that information in different ways.

Generated Knowledge Prompting asks the model to generate relevant information from what it knows.

Retrieval-Augmented Generation retrieves relevant information from an external knowledge source.

9.5 When to Use RAG

Situation	Why RAG helps
Questions that need up-to-date facts	External knowledge can be refreshed easily
Domain-specific or specialized topics	You can connect the model to your own documents
High need for factual accuracy	Retrieved evidence reduces unsupported claims
Applications where knowledge changes	Avoids expensive full-model retraining

RAG is less necessary for pure reasoning tasks, creative writing, or questions the model already answers reliably from its training data.

9.6 Try It Yourself

Task: Design a RAG-infused prompt structure that forces an LLM to answer a question using only the retrieved context provided.

User Query:

```
What is our company's remote work policy regarding
international travel?
```

Retrieved Document Chunk:

```
Employees are permitted to work remotely from any location
within their home country for up to 90 days per calendar year.
International remote work is strictly restricted due to tax and
compliance regulations, requiring explicit VP approval at least
30 days in advance.
```

Create a prompt template that takes the retrieved document chunk and the query, and ensures the model does not hallucinate rules outside of the text.

9.6.1 Example Answer

RAG prompt:

```
You are a helpful HR assistant. Answer the user's question
using ONLY the provided context below. If the answer cannot be
found entirely within the context, state "I cannot answer this
based on current company policies." Do not assume or
extrapolate.

Context:
"Employees are permitted to work remotely from any location
within their home country for up to 90 days per calendar year.
International remote work is strictly restricted due to tax and
compliance regulations, requiring explicit VP approval at least
30 days in advance."

User Query:
"What is our company's remote work policy regarding
international travel?"

Answer:
```

Expected model response:

> International remote work is strictly restricted due to tax and compliance regulations. It requires explicit VP approval at least 30 days in advance.

9.7 What's Next

You've now learned how to connect a language model to external knowledge sources through Retrieval-Augmented Generation.

The key is to **retrieve relevant information, provide it as context, and use it to generate a grounded answer**.

But retrieving information is only one way to extend what a model can do. Some tasks also require calculations, database queries, or other tool executions as part of the solution.

That's where **Automatic Reasoning and Tool-use (ART)** comes in — a framework that combines multi-step reasoning with external tools to solve tasks more systematically.

10 Automatic Reasoning and Tool-use (ART)

You've learned how to break tasks into linear pipelines, explore branching thoughts, and ground models using external knowledge through RAG.

Many real tasks, however, also require using tools — calculators, search engines, code interpreters, or APIs. Manually writing demonstrations that interleave reasoning and tool use for every new task is time-consuming.

What if a model could automatically select the right tools and generate its own multi-step reasoning program in a zero-shot fashion?

This is the core idea behind **Automatic Reasoning and Tool-use (ART)**

10.1 What Is Automatic Reasoning and Tool-use (ART)?

ART is a framework that uses a frozen language model to automatically generate intermediate reasoning steps as a program, seamlessly interleaving multi-step reasoning with external tool calls.

Instead of writing custom scripts for every new task, ART leverages a task library containing task-specific demonstrations of reasoning and tool use. When given a brand-new problem, ART automatically selects relevant examples and guides the model through a structured execution loop.

> **Automated Interleaving :** ART bridges the gap between static text reasoning and dynamic tool execution, allowing language models to decide when to think, when to call a tool, and how to incorporate tool outputs back into the reasoning chain.

10.2 How ART Works

At a high level the process looks like this:

1. **Select demonstrations:** From a library of previous tasks, ART retrieves examples that show useful patterns of reasoning and tool use.
2. **Generate a reasoning program:** The model produces a sequence of steps, written in a form that can include tool calls.
3. **Pause and call tools:** When the generation reaches a tool call, the system executes the tool and feeds the output back into the model.
4. **Resume and finish:** The model continues from the tool result and produces the final answer.

10.3 Key Characteristics and Advantages

Feature	Manual Tool-Use Scripting	Automatic Reasoning and Tool-use (ART
Demonstrations	Hand-crafted for each specific task.	Automatically selected from a task library.
Extensibility	Hard to scale; requires rewriting code for new tools.	Highly extensible; humans can easily add new tools or fix reasoning steps in the library.
Generalization	Rigid execution paths.	Zero-shot generalization to unseen tasks across benchmarks like BigBench and MMLU.
Human-in-the-Loop	Difficult to intervene mid-execution.	Allows seamless human feedback to correct intermediate reasoning errors.

💡 **Extensibility via Libraries :** Because ART relies on a modular task and tool library, you don't need to retrain the underlying model to teach it new tricks. You simply update the library with examples of the new tool in action.

10.4 Try It Yourself

Task: Design a pseudo-code or step-by-step ART program template for a model that needs to calculate the distance between two cities and check current weather conditions using external tools.

Available Tools:

- **Calculator(expression):** Solves math expressions.
- **WeatherAPI(city):** Fetches current temperature.
- **MapDistance(city1, city2):** Returns distance in miles.

Create a structured prompt template that shows how ART interleaves text reasoning with these tool calls.

10.4.1 Example Answer

ART Program Template:

```
Task: Calculate travel time for a 60 mph road trip from Paris
to Brussels and check arrival weather.
```

```
Execution Program:
Step 1: [Reasoning] Identify the two locations needed for
distance calculation.
Step 2: [Tool Call] distance = MapDistance("Paris", "Brussels")
[Pause & Inject Output]: distance = 193 miles

Step 3: [Reasoning] Calculate travel time assuming an average
speed of 60 mph.
Step 4: [Tool Call] hours = Calculator(193 / 60)
[Pause & Inject Output]: hours = 3.21 hours

Step 5: [Reasoning] Check destination weather for packing
recommendations.
Step 6: [Tool Call] weather = WeatherAPI("Brussels")
[Pause & Inject Output]: weather = 14°C, rainy

Step 7: [Final Answer Synthesis] The drive is 193 miles, takes
about 3.2 hours at 60 mph, and the weather in Brussels is 14°C
and rainy. Pack an umbrella!
```

10.5 What's Next

You've now learned how **Automatic Reasoning and Tool-use (ART)** combines multi-step reasoning with external tool execution.

The key idea is to **select relevant demonstrations, generate intermediate steps, and incorporate tool outputs into the ongoing process**.

But building effective AI systems also depends on the instructions that guide their behavior. Manually writing and testing different prompts can become time-consuming.

That's where **Automatic Prompt Engineer (APE)** comes in — a framework that automatically generates, evaluates, and selects instructions to improve performance on a given task.

11 Automatic Prompt Engineer (APE)

You've learned how to design prompts manually — from writing clear instructions and few-shot examples to structuring multi-step reasoning, RAG pipelines, and tool execution.

However, manually tweaking prompt phrasing, searching for optimal words, and testing subtle variations by hand is time-consuming and often relies on trial-and-error.

What if language models could write and optimize their own prompts automatically?

This is the idea behind **Automatic Prompt Engineer (APE)**.

11.1 What Is APE?

Automatic Prompt Engineer (APE) is a framework that automates instruction generation and selection. It frames prompt engineering as a natural language synthesis and black-box optimization problem, using language models to generate, evaluate, and search for the best-performing instructions for a given task.

Instead of relying entirely on human intuition, APE uses a language model to propose candidate instructions and evaluates those candidates using a target model.

11.2 How APE Works

The APE optimization process follows a two-stage loop:

11.2.1 Stage 1: Candidate Instruction Generation

An inference model (LLM) is provided with input-output demonstrations of a task (e.g., pairs of math questions and their answers) and prompted to synthesize a high-level instruction that describes the task.

11.2.2 Stage 2: Evaluation and Selection

1. **Execution:** The candidate instructions are appended to test inputs and executed using a target language model.
2. **Scoring:** The outputs are scored using an evaluation metric (e.g., execution accuracy or log-likelihood).
3. **Filtering & Refinement:** APE filters out weak instructions and can further propose subtle variations on top-scoring candidates to find the most effective instruction.

One notable result from the paper is that APE discovered a zero-shot Chain-of-Thought prompt that outperformed the well-known human-written phrase "Let's think step by step."

The automatically found prompt was:

```
Let's work this out in a step by step way to be sure we have
the right answer.
```

11.3 Advantages and Limitations of APE

11.3.1 Why Use APE?

- **Less manual trial and error:** APE can generate and evaluate many candidate instructions without requiring a person to write each version.
- **Measurable selection:** Candidates can be compared using task-specific evaluation metrics.
- **Potential performance improvements:** Automatically generated instructions can outperform manually written alternatives on particular tasks.
- **Reusable optimization process:** The same general approach can be applied to different tasks by changing the examples, target model, and evaluation criteria.

11.3.2 What to Keep in Mind

APE does not guarantee that the selected prompt will be correct or effective in every situation.

- **Evaluation quality matters.** If the test examples or evaluation metric do not represent the real task, the selected instruction may optimize the wrong behavior.
- **Optimization can be expensive.** Generating and testing many candidates requires additional model calls, time, and computational resources.
- **Overfitting is possible.** A candidate might perform well on the examples used during selection but poorly on new inputs. Keeping a separate validation set helps identify this problem.
- **Results are model-dependent.** An instruction that works well with one model may not perform equally well with another.
- **Human review can still be useful.** Evaluation scores alone may not capture every important requirement, such as clarity, safety, consistency, or the correct handling of ambiguous inputs.

APE is therefore best understood as a way to improve prompt discovery and selection, not as a replacement for evaluation and human judgment.

11.4 Try It Yourself

Task: Simulate the APE instruction-generation stage manually by acting as the inference model.

Input-Output Demonstrations:

Review	Expected Output
The package was delivered late and damaged. Score: 1/5	NEGATIVE
Setup was straightforward and support was very helpful. Score: 5/5	POSITIVE
he item works as expected, nothing special. Score: 3/5	NEUTRAL

Design a meta-prompt that asks an LLM to generate 3 candidate instructions that explain how to map these inputs to outputs.

11.4.1 Example Answer

Meta-Prompt for Candidate Generation:

```
You are an expert prompt optimization engine. Below are
input-output demonstrations of a task.

Demonstrations:
- Input: "The package was delivered late and damaged. Score:
1/5" -> Output: Negative
- Input: "Setup was straightforward and support was very
helpful. Score: 5/5" -> Output: Positive
- Input: "The item works as expected, nothing special. Score:
3/5" -> Output: Neutral

Task: Generate 3 distinct, concise instruction candidates that
could be placed at the top of a prompt to guide a language
model to perform this task accurately.

Instructions:
```

Model's Generated Candidate Instructions:

> Candidate A: "Analyze the customer review text and numerical score, then classify the sentiment as Positive, Neutral, or Negative."
> Candidate B: "Assign a sentiment label (Positive/Neutral/Negative) based on the customer's satisfaction level indicated in the review."
> Candidate C: "Determine the overall emotion of the feedback, using the 1-5 rating as supporting context."

(An APE framework would then test these 3 candidates against a validation set to select the winner!)

11.5 What's Next

You've now learned how **Automatic Prompt Engineer (APE)** can generate and evaluate candidate instructions to discover which prompts work best for a particular task.

The key is to **treat prompt design as an optimization process rather than relying entirely on manual trial and error**.

But choosing better instructions doesn't necessarily solve every reasoning problem. Sometimes, the model struggles with specific questions, and human-written examples could help — if we knew which questions were worth annotating.

That's where **Active-Prompt** comes in — a technique that uses model uncertainty to identify the most informative questions for human annotation.

12 Active-Prompt

You've learned how techniques like Few-Shot CoT rely on human-annotated step-by-step examples to guide language models through complex reasoning tasks.

However, standard few-shot methods use a fixed, static set of examples chosen arbitrarily by humans. The problem is that these fixed examples might not be the most effective demonstrations for every task, nor do they address the specific edge cases where the model struggles most.

How can we automatically identify which questions a model is most confused about so we only annotate the most helpful examples?

This is the idea behind **Active-Prompt**: use the model's uncertainty to decide which examples deserve human attention.

12.1 What Is Active-Prompt?

Active-Prompt is an adaptive prompting framework that uses uncertainty estimation to select the most informative questions for human annotation. Inspired by active learning in traditional machine learning, it ensures that human effort is spent annotating only the examples that provide the highest value to the model.

Instead of randomly choosing or hand-picking a fixed set of exemplars, the approach:

1. Lets the model answer a pool of training questions multiple times
2. Measures how uncertain the model is on each question
3. Asks humans to annotate only the most uncertain questions with step-by-step reasoning
4. Uses those newly annotated examples as the final CoT demonstrations

12.2 How Active-Prompt Works

The process can be summarized in four steps:

1. **Generate multiple answers:** For each training question, the model produces k different answers (with or without existing CoT examples).
2. **Measure uncertainty:** An uncertainty score is calculated for each question. A common way is to look at disagreement among the k answers — the more the answers differ, the higher the uncertainty.
3. **Select questions for annotation:** The questions with the highest uncertainty are chosen for human annotation.
4. **Use the new exemplars:** Humans write Chain-of-Thought reasoning for the selected questions. These annotated examples are then used to prompt the model on the actual test questions.

The key idea : Active-Prompt uses **model uncertainty to guide human annotation**. Instead of choosing examples randomly or relying only on intuition, it prioritizes questions where the model's answers disagree.

12.3 A Simple Example

Imagine you're developing an AI assistant for a business that answers numerical questions about sales and inventory.

You have a collection of 100 questions but only enough time to annotate a small number of them manually.

You could randomly select 10 questions and write detailed Chain-of-Thought explanations for each one.

Alternatively, you could use an Active-Prompt-style approach.

12.3.1 Without Active-Prompt

1. Select 10 questions.
2. Write human explanations for those questions.
3. Add the examples to the prompt.
4. Evaluate the model's performance.

The selected questions might be easy ones that the model already solves reliably.

12.3.2 With Active-Prompt

1. Generate multiple answers for the questions in the pool.
2. Measure the disagreement between answers.
3. Select the questions with the highest estimated uncertainty.
4. Ask a human to annotate their solutions.
5. Include the annotated examples in the prompt.
6. Evaluate the model on a separate set of questions.

This approach directs annotation effort toward questions where the model's behavior appears less reliable.

The difference is not that Active-Prompt removes the need for human expertise. Instead, **it helps decide where that expertise should be applied**.

12.4 When to Use Active-Prompt

Situation	Why Active-Prompt helps
You are building CoT prompts for a new task	Helps find the most informative examples to annotate
Annotation budget is limited	Directs human effort to the questions that matter most
Model performance varies a lot across examples	Uncertainty scoring identifies difficult cases
You want better task adaptation	Produces exemplars more suited to the target task

12.5 Try It Yourself

Task: Simulate the uncertainty selection phase of Active-Prompt to identify which question needs human CoT annotation.

Scenario: We queried an LLM 4 times for three different math questions. Below are the generated final answers for each run.

Question	1	2	3	4
What is 12 x 15?	180	180	180	180
A store gives a 20% discount on a $50 shirt, then adds 8% tax. What is the final price?	43.20	43.20	40.00	43.20
A train leaves Station A at 60 mph. An hour later, a second train leaves at 80 mph. How long until the second train catches up?	3 hours	4 hours	2.5 hours	3 hours
A customer buys 3 items at 12 DT each and receives a 5 DT discount. What is the final price?	31 DT	36 DT	31 DT	

Identify which question has the highest uncertainty and requires a human-annotated CoT exemplar.

12.5.1 Example Answer

Uncertainty Analysis:

> Question 1: 100% agreement (180). Zero Uncertainty.
> Question 2: 75% agreement (43.20 vs 40.00). Low Uncertainty.
> Question 3: High disagreement (3 hours, 4 hours, 2.5 hours). Highest Uncertainty.

Selection: Question 3 should be selected for human annotation.

Human Annotation Step:

> The human annotator creates a step-by-step exemplar for Question 3:
> 1. Train 1 travels 60 miles in the first hour.
> 2. The relative speed closing the gap is 80 - 60 = 20 mph.
> 3. Time to close 60 miles: 60 / 20 = 3 hours.

Adding this exemplar to the prompt will help the model master relative speed problems moving forward!

12.6 What's Next

You've now learned how **Active-Prompt** uses uncertainty to identify difficult questions and direct human annotation toward the examples most likely to help.

The key is to **spend annotation effort where the model needs the most guidance**, rather than treating every example equally.

But sometimes, the challenge isn't selecting better examples. It's guiding a model toward particular information, keywords, or aspects of an input without changing the main model's parameters.

That's where **Directional Stimulus Prompting (DSP)** comes in — a framework that generates targeted hints to steer a language model toward the desired output.

13 Directional Stimulus Prompting (DSP)

You've learned how to optimize prompts automatically using frameworks like APE, select high-value annotations with Active-Prompt, and ground model responses using external tools or RAG.

However, when working with large, black-box language models (like commercial LLMs that you cannot fine-tune directly), guiding them toward a specific, nuanced output — such as incorporating particular keywords, following subtle style constraints, or steering summarization focus — can be extremely difficult with static prompts.

How can you dynamically guide a frozen black-box model for every unique input instance without altering its internal weights?

This is the idea behind **Directional Stimulus Prompting (DSP)**.

13.1 What Is Directional Stimulus Prompting?

Directional Stimulus Prompting is a framework that uses a small tunable policy model to generate a "stimulus" (a short hint) which is then added to the prompt of a larger, frozen language model.

The process works as follows:

1. A small policy language model generates a directional hint based on the input.
2. The hint is inserted into the prompt given to the main (black-box) LLM.
3. The main LLM produces the final output, guided by both the original instruction and the hint.

The policy model can be optimized (for example with reinforcement learning) so that the hints it produces lead to better results from the main model.

> **The Policy-Model Approach :** Think of the small policy model as an expert coach whispering brief, targeted hints into the ear of a massive model, telling it what focus areas or keywords to emphasize in its response.

13.2 How Directional Stimulus Prompting Works

DSP connects stimulus generation, target-model inference, and evaluation in a learning process.

13.2.1 Step 1: Provide the Original Input

For example, a system might receive a business report containing information about revenue, costs, customer feedback, and future plans.

The system also needs a clear task, such as producing a concise summary.

13.2.2 Step 2: Generate a Directional Stimulus

The policy language model receives the input and generates a stimulus intended to guide the target model toward the desired output.

For a business report, the stimulus might be: revenue trends, profitability, operating costs, business risks

For another report, the stimulus could emphasize customer satisfaction, product quality, and delivery performance.

The policy model is trained to discover which stimuli are useful for the task. The hints are not simply selected randomly or manually rewritten for every input.

13.2.3 Step 3: Combine the Stimulus with the Input

The original input and generated stimulus are provided to the target model.

Conceptually, the target model receives something like:

```
Task:
Summarize the following business report.

Focus on these aspects:
revenue trends, profitability, operating costs, business risks

Report:
[Original report]
```

The target model uses the hints alongside the original report to generate its summary.

The example above illustrates how a stimulus can be incorporated into a prompt. It is not the exact format required by the original DSP implementation.

13.2.4 Step 4: Evaluate the Output

Next, evaluate the target model's output against the task's objectives.

For a summarization task, evaluation might consider whether the summary captures important information from a reference summary. Other measures can assess relevance, completeness, or human preferences.

The evaluation provides feedback about whether the generated stimulus helped produce a better result.

13.2.5 Step 5: Improve the Policy Model

The policy model can be optimized using supervised fine-tuning or reinforcement learning.

In a reinforcement-learning setup, the system uses rewards derived from the target model's output to update the policy model. Stimuli that lead to better results can therefore receive stronger reinforcement through the optimization process.

Over repeated training iterations, the policy model learns to generate more useful hints for different inputs.

13.3 Directional Stimulus Prompting vs. Standard Prompting

Aspect	Standard Prompting	Directional Stimulus Prompting
Guidance	Relies on static instructions or manual cues.	Uses instance-specific, dynamic hints generated automatically.
Model Modification	None (static prompt text).	Small policy model is tuned via RL/SFT; target LLM stays frozen.
Flexibility	Rigid across different inputs; same prompt applies to all.	Highly adaptive; hints change per input instance to target specific weaknesses.

13.4 When to Use Directional Stimulus Prompting

Situation	Why it helps
You want better control over a frozen LLM	Hints steer the model without changing its weights
Instance-specific guidance is useful	Each input can receive its own tailored stimulus
You can train a small policy model	Optimization improves the quality of the generated hints
Summarization or generation tasks	Originally shown effective for guiding summary style and content

This approach is more involved than simple manual prompting, so it is most appropriate when the performance gain justifies the extra system complexity.

13.5 Try It Yourself

Task: Simulate a Directional Stimulus Prompt manually.

Suppose you want an LLM to summarize a long news article about renewable energy adoption, but you need to make sure the summary explicitly highlights two specific sub-topics: ”government subsidies” and ”supply chain bottlenecks”.

Design a directional stimulus prompt template that accepts an article text along with these generated hints, steering the frozen LLM to integrate them naturally.

13.5.1 Example Answer

DSP-style template:

```
Article Text:
{{ article_text }}

Directional Hint / Focus Keywords:
- government subsidies
- supply chain bottlenecks

Instructions:
Write a concise 3-sentence summary of the article. You MUST
naturally incorporate both directional hint topics into your
summary. Do not list them out; weave them into the narrative
flow.

Summary:
```

13.6 What’s Next

You’ve now learned how **Directional Stimulus Prompting (DSP)** uses generated hints to guide a language model toward specific aspects of an input or desired output.

The key is to **adapt the guidance to each input instead of relying only on a fixed prompt**.

However, better instructions and targeted hints cannot eliminate every computational error. When a task involves complex calculations or precise date arithmetic, a language model may still produce an incorrect result.

That’s where **Program-Aided Language Models (PAL)** come in — a technique that lets a model generate a program and delegate the actual computation to an execution environment.

14 Program-Aided Language Models (PAL)

You've learned how to guide models using step-by-step reasoning, external tools, and automated pipelines.

However, even with Chain-of-Thought (CoT) prompting, large language models are fundamentally text predictors — not calculators. When faced with complex arithmetic, multi-step date calculations, or precise logic, LLMs frequently make simple math errors or hallucinate numbers because they generate answers token-by-token rather than executing actual math.

What if the model could translate the question into a small Python program and let a computer execute that program to calculate the answer?

This is the idea behind **Program-Aided Language Models (PAL)**.

14.1 What Is Program-Aided Language Models (PAL)?

Program-Aided Language Models (PAL) is a technique that uses LLMs to read natural language problems and generate computer programs (such as Python code) as intermediate reasoning steps.

Instead of using free-form text to arrive at a solution (like traditional CoT), PAL offloads the actual calculation and execution step to an external programmatic runtime (a Python interpreter).

14.2 How Program-Aided Language Models Works

The process is straightforward:

1. The model receives a natural language question (often with a few examples).
2. It generates a program that solves the question.
3. The program is executed in an interpreter.
4. The result of the execution is returned as the answer.

Example question:

```
Today is 27 February 2023. I was born exactly 25 years ago.
What is the date I was born in MM/DD/YYYY?
```

Model-generated program (simplified):

```
from datetime import datetime
from dateutil.relativedelta import relativedelta

today = datetime(2023, 2, 27)
```

```
born = today - relativedelta(years=25)
print(born.strftime('%m/%d/%Y'))
```

Execution result:

```
02/27/1998
```

By letting the interpreter handle the date arithmetic, PAL avoids common language-model mistakes in calculation.

14.3 Advantages and Limitations of PAL

14.3.1 Why Use PAL?

- **More reliable computation:** A programming runtime handles supported calculations directly instead of requiring the model to perform every operation through generated text.
- **Clear separation of responsibilities:** The LLM interprets the problem and generates the program; the runtime executes it.
- **Flexible problem solving:** The approach can support mathematical, symbolic, date-related, and algorithmic tasks that can be represented in code.
- **Inspectable intermediate representation:** The generated program can be examined to understand which operations the application will execute.
- **Integration with existing tools:** Programs can use suitable libraries and computational systems when these are available in the execution environment.

14.3.2 An Important Security Consideration

Generated code must be treated as untrusted input.

A simple example can illustrate code execution using `exec()`, but **directly executing unrestricted model-generated code in a production application is unsafe**.

A malicious or incorrectly generated program could access files, consume excessive resources, or perform unintended operations.

A production system should use an appropriately restricted execution environment, validate allowed operations, impose resource limits, and prevent access to sensitive data and unnecessary system capabilities.

The exact safeguards depend on the application and runtime being used.

14.4 Try It Yourself

Task: Imagine you are building a PAL snippet to solve a financial calculation problem.

```
An employee starts with a base salary of $50,000. They receive
a 5% raise each year for 3 years. What is their salary at the
end of Year 3?
```

Write a Python code snippet using basic variables or a loop that an LLM would generate for a PAL pipeline to solve this accurately.

14.4.1 Example Answer

Generated PAL Code Snippet:

```python
# Base salary calculation with compounding annual raises
salary = 50000
raise_rate = 0.05
years = 3

for year in range(years):
    salary = salary * (1 + raise_rate)

# Format the final answer
print(f"${salary:.2f}")
```

Runtime Output:

```
57881.25
```

14.5 What's Next

You've now learned how **Program-Aided Language Models (PAL)** combine natural-language understanding with executable programs to solve computational problems.

The key is to **let the model interpret the problem and generate code while an execution environment handles the computation**.

But some tasks require more than executing a single program. They involve gathering information, observing results, and deciding what action to take next based on what happens.

That's where **ReAct (Reasoning and Acting)** comes in — an approach that interleaves reasoning, actions, and observations to help models solve tasks through interaction with external tools and environments.

15 ReAct Prompting

You've learned how to guide models through step-by-step text reasoning, ground them with external facts using RAG, and even let them write code execution scripts using PAL.

However, what happens when a task requires a continuous, dynamic loop where a model needs to think, search the real world, observe the results, update its plan, and search again before finally answering?

This synergy between thinking and doing is captured in **ReAct (Reasoning and Acting)**.

15.1 What Is ReAct?

ReAct is a prompting framework that instructs language models to generate both verbal reasoning traces ("Thoughts") and task-specific actions in an interleaved manner.

The central idea is that reasoning and acting can support one another:

- **Reasoning** helps the model understand the problem, plan its next step, interpret observations, and adjust its approach.
- **Acting** allows the model to interact with external tools and environments to retrieve information or perform operations.

After each action, the model receives an **Observation** and uses it to update its reasoning. This loop continues until the model decides it has enough information to answer.

The combination of reasoning and acting helps the model create better plans, recover from mistakes, and ground its answers in external information.

15.2 How ReAct Works

ReAct commonly represents a task-solving process using three elements: **Thought, Action, and Observation**.

These elements repeat until the system has enough information to complete the task.

15.2.1 Step 1: Thought

The model identifies what it needs to accomplish and determines a useful next step.

A thought can help the model break a problem into smaller tasks, choose an appropriate tool, and adapt when new information becomes available.

15.2.2 Step 2: Action

The model selects an action that interacts with an available tool or environment.

An action might involve:

- Searching the web.
- Looking up information in a database.
- Calling an API.
- Running a calculation.
- Interacting with an application.
- Performing an operation in a simulated environment.

15.2.3 Step 3: Observation

After the action executes, the environment returns an observation.

An observation contains the result of the action, such as a search result, database record, calculation, or status message.

15.2.4 Step 4: Repeat the Cycle

The model interprets the observation and determines what to do next.

The system then executes another action and receives another observation.

This cycle can repeat as necessary. The model may change its plan when a tool returns unexpected results or when additional information is required.

15.2.5 Step 5: Produce the Final Answer

Once sufficient information has been collected, the model produces its answer.

> **ReAct vs. ART :**
> ART also combines reasoning and tool use. It emphasizes selecting demonstrations from a task library and using them to guide the model's reasoning and tool calls.
> ReAct emphasizes the interleaving of reasoning, actions, and observations during task execution.
> The techniques overlap conceptually, but their frameworks and implementation details differ.

15.3 Try It Yourself

Task: Write a short ReAct-style trajectory for this question. You can use a Search action and a Calculator action.

```
Who directed the movie Inception, and what is the square root
of the year it was released?
```

15.3.1 Example Answer

ReAct-style trajectory:

```
Thought 1: I need to find the director of Inception and the
year it was released.
Action 1: Search[Inception director and release year]
Observation 1: Inception was directed by Christopher Nolan and
released in 2010.

Thought 2: Now I need the square root of 2010.
Action 2: Calculator[sqrt(2010)]
Observation 2: 44.833

Thought 3: I have both pieces of information.
Action 3: Finish[Christopher Nolan directed Inception. The
square root of 2010 is approximately 44.83.]
```

15.4 What's Next

You've now learned how **ReAct (Reasoning and Acting)** combines reasoning with actions and observations in an iterative problem-solving loop.

The key is to **use feedback from each action to determine what to do next**, rather than following a fixed sequence of steps.

But even when an agent interacts with its environment, it can repeat the same mistakes across multiple attempts. How can it use feedback from previous failures to improve its next attempt?

That's where **Reflexion** comes in — a framework that uses verbal self-reflection and memory to help agents learn from previous experiences without updating their model weights.

16 Reflexion

You've learned how ReAct allows a language model to interleave reasoning and actions. However, when the model makes a mistake, it usually starts the next attempt from scratch.

What if we can add a self-improvement loop. After each attempt, the model reflects on what went wrong and stores that linguistic feedback in memory. On the next try, it uses this reflection to avoid repeating the same mistakes.

This is the idea behind **Reflexion**.

16.1 What Is Reflexion?

Reflexion is a framework that reinforces language-based agents through linguistic feedback (often called verbal reinforcement learning).

Instead of updating model weights via traditional scalar rewards, Reflexion converts environment feedback (success, failure, errors, or test results) into verbal self-reflection. This self-reflection is stored in the agent's memory and provided as context in subsequent trials, allowing the agent to rapidly and effectively learn from prior mistakes.

16.2 How Reflexion Works

Reflexion consists of three main components:

1. **Actor**

Generates actions and reasoning (often using CoT or ReAct). It produces a trajectory by interacting with the environment.

1. **Evaluator**

Scores the trajectory. The score can come from a rule-based heuristic, an LLM judge, or task-specific feedback.

1. **Self-Reflection**

Turns the evaluation and the trajectory into linguistic feedback. This reflection is stored in memory and used to guide the next attempt.

The typical cycle is:

1. Generate a trajectory (Actor)
2. Evaluate the result (Evaluator)
3. Produce a self-reflection
4. Store the reflection in memory
5. Generate a new trajectory that takes the reflection into account

Over multiple trials, the agent improves by learning from its own past mistakes in natural language form.

16.3 When to Use Reflexion

Situation	Why Reflexion helps
Tasks that benefit from trial and error	The agent can improve across multiple attempts
Fine-tuning is impractical	Learning happens through memory, not weight updates
Nuanced feedback is useful	Verbal reflections are more specific than scalar rewards
Interpretability matters	Reflections provide an explicit record of what the agent learned

Reflexion has shown strong results on sequential decision-making (AlfWorld), multi-hop reasoning (HotPotQA), and code generation (HumanEval, MBPP).

16.4 Try It Yourself

Suppose an agent tried to answer a multi-hop question and failed because it searched for the wrong entity.

Task: Write a short self-reflection the agent could store in memory to improve on the next attempt.

16.4.1 Example Answer

```
I failed because I searched for the wrong person.
Next time I should first confirm the full name and role of the
entity mentioned in the question before searching for related
facts.
```

On the next trial, this reflection would be added to the prompt so the agent can avoid the same mistake.

16.5 What's Next

You've now learned how **Reflexion** helps language-model agents improve across repeated attempts by evaluating their results, generating verbal feedback, and storing reflections in memory.

The key is to **turn experience into reusable feedback that can guide future attempts**.

So far, however, we've focused primarily on tasks where the essential information can be represented in text. What happens when solving a problem also requires interpreting an image, diagram, or other visual input?

That's where **Multimodal Chain-of-Thought Prompting** comes in — an approach that combines visual information and language-based reasoning to help models solve tasks involving multiple modalities.

17 Multimodal Chain-of-Thought Prompting

You've learned how to guide models using text-based Chain-of-Thought (CoT) reasoning, ReAct loops, and verbal reflection.

What happens when a question requires a model to interpret an image, read a scientific chart, solve a geometry diagram, or analyze a visual artifact alongside text? If you strip away the visual context, text-only reasoning collapses.

How can we bridge the gap between visual perception and language reasoning in prompts?

This is the core idea behind **Multimodal Chain-of-Thought Prompting**.

17.1 What Is Multimodal CoT

Multimodal CoT is a prompting and modeling approach that incorporates both language and visual inputs into the reasoning process.

Instead of generating an answer directly from mixed text and image inputs, it separates the work into two stages:

1. **Rationale Generation** — The model produces a step-by-step explanation using both the text and the visual information.
2. **Answer Inference** — The model uses the generated rationale (along with the original inputs) to produce the final answer.

This separation helps the model build a clearer intermediate representation before committing to an answer.

17.2 How Multimodal CoT Works

17.2.1 Step 1: Process Textual and Visual Information

The model receives the question together with the relevant image.

For example:

Question:

A 2 kg object rests on a frictionless incline. Using the angle shown in the diagram, calculate the component of gravitational force acting parallel to the incline.

Image:

A diagram showing a block on a frictionless inclined plane. The angle between the plane and the horizontal is 30°.

To solve the problem, the model needs to use information from both sources:

- The text provides the object's mass and describes the required calculation.
- The diagram provides the incline angle.

Neither the text nor the visual information necessarily provides everything needed

by itself.

17.2.2 Step 2: Generate a Rationale

An illustrative rationale might be:

```
The diagram shows an incline angle of 30 degrees.

The object has a mass of 2 kg, and gravitational acceleration
is approximately 9.8 m/s².

The gravitational force has magnitude:

F = mg = 2 × 9.8 = 19.6 N.

The component of gravity parallel to the incline is:

F_parallel = mg × sin(θ).

Using θ = 30 degrees:

F_parallel = 19.6 × sin(30°)
           = 19.6 × 0.5
           = 9.8 N.
```

The rationale connects visual observations with information from the question and the relevant physical relationship.

17.2.3 Step 3: Perform Answer Inference

Conceptually, the second stage receives:

- The original question.
- The original image.
- The rationale generated during the first stage.

The model uses these together to determine the requested result.

17.2.4 Step 4: Return the Final Answer

The final response should communicate the result clearly and include an appropriate explanation when useful.

17.3 Try It Yourself

Suppose you have an image of a simple circuit diagram and the following question:

```
In the circuit shown, if the switch is closed, will the light
bulb turn on?
Explain your reasoning.
```

Task: Write a two-stage Multimodal CoT-style prompt that first asks for a rationale and then asks for the final answer.

17.3.1 Example Answer

Stage 1 – Rationale Generation:

```
Look at the circuit diagram and the question.
Generate a step-by-step explanation of how the circuit works
and what happens when the switch is closed.
Use both the visual information and the question.
```

Stage 2 – Answer Inference:

```
Using the circuit diagram, the question, and the rationale
below,
give a clear final answer (Yes or No) and a short confirmation
of your reasoning.

Rationale:
[insert generated rationale here]
```

17.4 What's Next

Multimodal CoT shows how chain-of-thought reasoning can be extended beyond text to include visual information.

You now have techniques that cover text-based reasoning, tool use, retrieval, program generation, self-improvement, and multimodal reasoning.

In practice, match the method to the input: use standard CoT for text-only problems, and multimodal approaches when images or diagrams are essential to solving the task.

18 Introduction to AI Agents

Throughout this guide, you have learned how to write effective prompts, guide step-by-step reasoning, chain tasks together, and integrate external knowledge and tools.

Now, we take the ultimate step in prompt engineering and system design: AI Agents.

While traditional LLMs excel at simple, narrow tasks like text translation or writing short emails, they struggle when faced with broad, open-ended goals that require planning over time, searching live environments, and recovering from errors.

By combining large language models with planning, memory, and tool usage, we transform static text generators into **autonomous AI Agents**.

18.1 What Is an AI Agent?

An **AI Agent** is an LLM-powered system designed to perceive its environment, formulate plans, take actions, and accomplish complex goals autonomously with minimal human intervention.

Unlike a standard language model that only generates text, an agent typically includes:

- **Planning and reflection**
- **Tool access**
- **Memory**

18.2 Core Components of an AI Agent

An autonomous agent architecture typically consists of three foundational building blocks built around the core LLM "brain":

18.2.1 Planning and Reflection

- **Decomposition:** The agent breaks down a complex high-level goal into manageable sub-goals (using techniques like Chain-of-Thought, Prompt Chaining, or Tree of Thoughts).
- **Reflection & Self-Correction:** The agent evaluates past actions, analyzes errors (using frameworks like Reflexion), and adjusts its plan for future steps.

18.2.2 External Tool Access

Standalone LLMs are limited by their static training data. Agents can call external tools to interact with the real world, including:

- Web search engines and web scrapers

- Code interpreters and Python runtimes (like PAL)
- Internal company databases and Vector DBs (via RAG)
- External APIs (for sending emails, creating calendar events, or placing orders)

18.2.3 Memory Systems

- **Short-Term Memory:** In-context learning provided by recent conversation turns and immediate prompt history.
- **Long-Term Memory:** External vector storage or databases that allow agents to retain, retrieve, and recall facts, user preferences, and past experiences across multiple sessions.

18.3 Why Build with Agents

Standalone language models work well for narrow tasks such as translation, summarization, or drafting an email. They struggle when a task requires:

- Multiple sequential steps
- Access to up-to-date or private information
- Interaction with external systems
- Adjustment based on intermediate results

For example, creating a marketing strategy may involve researching competitors, analyzing market data, checking internal company metrics, and then synthesizing a plan. A single prompt is rarely enough.

Agents address this by combining the reasoning ability of an LLM with planning, tools, and memory.

18.4 Common Use Cases for AI Agents

AI Agents are revolutionizing modern software across every major sector:

- **In-Depth Research & Reporting:** Agents that automatically scour academic papers, summarize findings, verify data points, and draft comprehensive report documents.
- **Autonomous Software Development:** Agents that read bug reports, inspect codebase files, write code fixes, run test suites, and create pull requests.
- **Customer Support & Service:** Specialized agents that verify user identities, check order statuses in backend databases, issue refunds, or escalate issues when necessary.
- **E-Commerce & Event Booking:** Personal assistants that manage travel arrangements, search flights, handle hotel reservations, and organize complex itineraries.

- **Financial Analysis:** Systems that ingest market trends, pull SEC filings, run mathematical risk models via code execution, and deliver investment briefs.

18.5 Try It Yourself

Think of a task you regularly perform that requires several steps and external information (for example, planning a trip or researching a purchase).

Task: Briefly describe:

1. What the overall goal is
2. What tools or information sources would be needed
3. What kind of memory or intermediate results the agent should keep track of

18.5.1 Example Answer

Goal: Plan a weekend trip to a nearby city.

Tools / sources needed:

- Weather forecast
- Train or flight search
- Hotel booking site
- Restaurant recommendations

Memory / intermediate results:

- Preferred dates and budget
- Shortlist of neighborhoods
- Transportation options already checked
- Final itinerary being built

18.6 What's Next

You've now learned what AI agents are, how they differ from basic LLM interactions, and why they're useful for tasks involving multiple steps, external tools, and changing information.

The key idea is that **an AI agent combines an LLM with the capabilities needed to work toward a goal, observe results, and decide what to do next**.

But how do these capabilities fit together inside an agent?

Next, we'll explore **Agent Components** to understand the main building blocks of an AI agent and how they work together to complete tasks.

19 AI Workflows vs. AI Agents

You've learned that AI agents can combine language models, tools, memory, and planning to complete complex tasks.

But here's an important question: **Do you always need an AI agent to automate a task?**

Imagine you're building an AI-powered customer support system. When a customer asks about a refund, the system must classify the request, retrieve the relevant policy, and generate a response.

You could design a fixed sequence of steps that handles every request according to predefined rules. Or, you could build an AI agent that decides which tools to use and what actions to take based on each customer's situation.

Both approaches can work, but they offer different levels of control and flexibility.

In this guide, you'll learn the difference between **AI workflows** and **AI agents**, explore common workflow patterns, and discover when each approach makes the most sense.

19.1 What Are Agentic Systems

Agentic systems can be divided into two main categories:

1. **AI Workflows** — LLMs and tools are orchestrated through predefined steps and explicit control flow.
2. **AI Agents** — The LLM dynamically decides what to do next, which tools to use, and how to complete the task.

Both can be powerful. The right choice depends on how structured or open-ended the task is.

19.2 AI Workflows

AI workflows follow a fixed sequence of steps defined in advance. The path is explicit and predictable.

Key characteristics:

- Predefined steps and execution paths
- High predictability and control
- Clear task boundaries
- Explicit orchestration logic

When to use workflows:

- The task is well-defined
- Consistency and reliability are critical

- You need full control over every step
- The process rarely changes

Common patterns:

- **Prompt chaining** — Break a task into sequential LLM calls
- **Routing** — Send different types of requests to specialized paths
- **Parallelization** — Run independent steps at the same time

19.3 AI Agents

AI agents give the language model more autonomy. The model decides which tools to use, in what order, and when the task is finished.

Key characteristics:

- Dynamic decision-making
- Autonomous tool selection
- Reasoning and reflection
- Self-directed execution

When to use agents:

- The task is open-ended
- The number of steps is hard to predict in advance
- Flexibility is more important than strict control
- The system needs to adapt based on intermediate results

19.4 Key Differences

Aspect	AI Workflows	AI Agents
Control Flow	Predefined and explicit	Dynamic and autonomous
Decision Making	Hard-coded logic	LLM-driven reasoning
Tool Usage	Orchestrated by code	Selected by the agent
Adaptability	Fixed paths	Flexible execution
Predictability	High	Lower
Best for	Well-defined tasks	Open-ended problems

19.5 Choosing the Right Approach

Prefer workflows when:

- Requirements are clear and stable

- You need strong predictability
- Debugging and monitoring are priorities
- Cost and reliability must be tightly controlled

Prefer agents when:

- The problem is exploratory or complex
- The exact sequence of steps cannot be fully defined upfront
- Adaptive reasoning adds clear value
- Flexibility outweighs the need for strict control

Many real systems combine both: workflows provide structure, while agents handle the more open-ended parts.

19.6 Try It Yourself

Consider these two tasks:

1. Generate a product description from a structured list of features, then translate it into three languages.
2. Research a new market, decide which data sources are relevant, gather information, and produce a short strategy recommendation.

Task: Decide which one is better suited for a workflow and which one is better suited for an agent. Briefly explain why.

19.6.1 Example Answer

Task 1 → Workflow The steps are clear and sequential (generate → translate). A predefined chain is reliable and easy to control.

Task 2 → Agent The research process is open-ended. The system needs to decide what to search, which sources to trust, and when it has enough information. Autonomy is more useful here.

19.7 What's Next

You now understand the core distinction between AI workflows and AI agents.

In the following sections we will look more closely at the building blocks of agents — including planning, tool use, memory, and common design patterns — so you can decide when and how to apply each approach.

20 Context Engineering for AI Agents

When building AI agents, the quality of the results often depends less on the model itself and more on the context you give it.

Context engineering is the practice of carefully designing the instructions, constraints, tool descriptions, and memory that guide an agent's behavior.

20.1 What Is Context Engineering

Context engineering goes beyond writing a single good prompt. For agents it usually includes:

- System prompts that define the agent's role and rules
- Clear constraints on what the agent must or must not do
- Well-written tool descriptions
- Memory and state management across steps
- Error-handling guidance

The goal is to make the agent's behavior more reliable, consistent, and aligned with what you actually want.

20.2 Why Context Engineering Matters

Agents make many decisions on their own. Without clear context they may:

- Skip steps without explanation
- Use tools incorrectly
- Produce incomplete results
- Behave inconsistently across similar tasks

Good context reduces these problems by making expectations explicit and giving the agent useful guidance at every stage.

20.3 Common Problems and Fixes

Problem	Typical Cause	Better Approach
Agent skips parts of the task	Instructions are too vague	Require explicit completion or justification
Unpredictable behavior	Ambiguous rules	Make required actions and formats explicit
Hard to debug	No visibility into decisions	Log steps, status, and reasoning

Continued on next page

Problem	Typical Cause	Better Approach
Overly rigid behavior	Too many strict rules	Allow justified flexibility
Failure on edge cases	No error-handling guidance	Add clear fallback instructions

20.4 Practical Principles

Be explicit Don't assume the agent knows what "good" looks like. State required steps, output format, and quality criteria.

Make expectations visible Tell the agent what it must do, what it can decide, and how to report its decisions.

Add observability Track tasks, tool calls, and status so you can see where things go wrong.

Iterate from real behavior Run the agent, observe failures, and refine the context. Context engineering is an ongoing process.

Balance control and flexibility Too many rigid rules make the agent brittle. Too few rules make it unreliable. Aim for clear defaults with room for justified exceptions.

20.5 Try It Yourself

Suppose you are building a research agent that should:

1. Break a topic into several search subtasks
2. Run a search for each subtask
3. Write a short report

Task: Write a short system-prompt rule that prevents the agent from silently skipping any of the planned searches.

20.5.1 Example Answer

```
For every search subtask you create, you must either:
1. Execute the search and record the findings, or
2. Explicitly mark the task as skipped and explain why it is
unnecessary.

Never skip a planned search without a written justification.
```

20.6 What's Next

Context engineering is one of the most important skills when building reliable agents.

In the following sections we will look at other core building blocks — including planning strategies, tool design, and memory — and how they work together with good context.

21 Context Engineering Deep Dive: Building a Deep Research Agent

You've learned how context engineering helps AI agents follow instructions, use tools, manage information, and handle unexpected situations.

But knowing these principles is only the beginning. **How do you apply them when building an agent that performs multiple searches, manages tasks, and produces a complete research report?**

Imagine building a deep research agent. It receives a question, plans several searches, collects information, and combines its findings into a report.

Your first version might work well on simple questions but fail on more complex ones. It might skip searches, forget to update task statuses, or produce incomplete reports.

Improving the system requires more than repeatedly adding instructions to the system prompt. You must also examine the agent's architecture, tool definitions, communication between components, and execution state.

In this guide, you'll explore these challenges by developing a deep research agent step by step.

21.1 Why Context Engineering Requires Iteration

When building an AI agent, your first implementation is rarely perfect.

Even if your initial system prompt looks clear, the agent may interpret instructions differently than expected. Tools may return unexpected results, and the amount of information accumulated during execution may become difficult to manage.

Building a reliable agent often requires repeatedly refining several components:

- **System prompts:** Defining the agent's responsibilities, rules, and expected behavior.
- **Tool definitions:** Explaining what each tool does, when it should be used, and what information it requires.
- **Agent architecture:** Deciding which responsibilities belong to which components.
- **Input and output formats:** Defining how information moves between tools and agents.
- **Execution tracking:** Ensuring that tasks, results, and failures are recorded consistently.

Consider a research agent that creates three search tasks but executes only two.

You might initially respond by adding another instruction to the system prompt. That could help, but it might not address the underlying problem.

Perhaps the agent is responsible for too many operations, its task tracker is difficult to manage, or it receives so much information that important requirements become harder to follow.

The key principle is to investigate why the system fails before deciding what to change.

Sometimes the solution is a clearer instruction. In other situations, you may need better tool descriptions, more explicit task tracking, or a different architecture.

21.2 Designing the Research Agent's Architecture

Let's start with the system we're trying to build.

The user provides a research question, and the system must investigate it using web searches before producing a report.

For example:

> Research the current challenges facing small businesses adopting cloud-based software.

The agent must understand the request, identify the main research questions, execute searches, track progress, and synthesize the results.

There are several ways to organize this process.

21.2.1 The Initial Design: One Agent Does Everything

In a simple implementation, a single agent handles the entire task.

It might be responsible for:

1. Planning the research.
2. Creating and updating search tasks.
3. Calling web search tools.
4. Saving findings to memory or a spreadsheet.
5. Checking which tasks remain incomplete.
6. Writing the final report.

This design is straightforward because one agent controls the entire process.

However, it also creates several potential problems.

Too many responsibilities

The agent must switch between planning, searching, tracking progress, and writing. Each responsibility introduces additional instructions and decisions.

Growing context

As the research progresses, the conversation may accumulate task descriptions, tool results, source summaries, and previous decisions.

Forgotten operations

The agent might collect useful search results but forget to update the task tracker. It might create a task without executing it or begin writing the report before all important questions have been investigated.

Inconsistent behavior

The agent might follow the intended process for one topic but behave differently when the research question becomes more complicated.

These problems do not mean that single-agent systems are inherently unreliable. For smaller tasks, one agent may be entirely sufficient. The difficulty arises when its responsibilities and accumulated context become too complex to manage reliably.

21.2.2 The Improved Design: Separate Planning from Searching

One way to improve the architecture is to introduce a dedicated **Search Worker Agent**.

Instead of requiring one agent to perform every operation, divide the responsibilities between two agents.

The **Deep Research Agent** acts as the orchestrator. It interprets the user's request, creates the research plan, delegates searches, tracks progress, and produces the final report.

The **Search Worker Agent** focuses on executing an individual search task and returning useful findings.

The overall process becomes:

1. The user submits a research question.
2. The Deep Research Agent identifies the research questions and creates a plan.
3. The agent records the planned tasks in the task tracker.
4. It sends each search task to the Search Worker Agent.
5. The worker performs the search and returns the findings.
6. The Deep Research Agent records the result and updates the task status.
7. The orchestrator verifies completion and prepares the final report.

This architecture separates planning and coordination from search execution.

21.2.3 Why Does This Help?

1. Clearer responsibilities

Each agent has a narrower role. The orchestrator can focus on planning and synthesis, while the worker concentrates on gathering relevant information.

2. Easier debugging

You can investigate whether a failure occurred during planning, search execution, result processing, or task tracking.

3. More focused context

The search worker does not need the complete conversation history or every detail of the research plan. It mainly needs the specific query and instructions required to execute it.

4. Flexible model selection

Different agents can use different models depending on their requirements. A reasoning-intensive orchestration task may justify a more capable model, while a straightforward search task may work well with a faster, less expensive model.

The ideal model depends on the task, quality requirements, cost, and available alternatives. Assigning a separate agent does not automatically improve performance; the additional coordination must provide enough value to justify its cost and complexity.

Important: A multi-agent architecture does not eliminate the need for good context engineering. It changes how you distribute responsibilities and information so that each component can perform its role more effectively.

21.3 Engineering the Main Agent's System Prompt

With the architecture defined, let's examine the instructions that guide the Deep Research Agent.

A useful system prompt should establish the agent's role, explain the main process, provide essential context, and describe how to use available tools.

21.3.1 Part 1: Define the Agent's Role

Start by describing what the agent is responsible for.

Example:

```
You are a deep research agent responsible for planning and
coordinating
research tasks to produce comprehensive, evidence-based
reports.

You create a research plan, delegate search tasks to the
available
Search Worker Agent, track execution, evaluate the findings,
and
synthesize the results into a final report.
```

This instruction establishes the agent's main responsibility.

Notice that it describes the orchestrator's role without requiring it to perform every search itself.

A clear role definition helps establish which operations belong to the main agent and which should be delegated.

21.3.2 Part 2: Define the Research Process

Next, explain how the agent should approach a research request.

Example:

```
GENERAL INSTRUCTIONS

1. Analyze the user's research question and identify the main
   information needs.

2. Create distinct search tasks that collectively address those
needs.

3. Record each task in the task tracker before executing it.

4. Delegate each search task to the Search Worker Agent.

5. Review the worker's response and record the findings.

6. Update the task status after processing the result.

7. Verify that all important research questions are addressed
or
   explicitly identified as unresolved.

8. Synthesize the findings into a structured research report.
```

The instructions describe the major stages without unnecessarily defining every internal decision.

However, the number of searches still needs to be specified according to your requirements.

A short factual query may need only one or two searches. A broad investigation may require several more. The objective is to create enough distinct tasks to cover the research question adequately.

If a particular application requires a fixed number of searches, make that requirement explicit rather than leaving it ambiguous.

21.3.3 Part 3: Provide Essential Context

Research depends on time-sensitive information. A query about the latest developments cannot be handled in exactly the same way as a question about historical events.

Providing the current date can help the agent interpret temporal expressions and choose relevant search periods.

For example, in a system that supports dynamic date injection:

```
For temporal context, today's date is:
{{ $now.format('yyyy-MM-dd') }}
```

This example uses an n8n expression to insert the current date into the prompt.

Other orchestration platforms may use different syntax.

Including the date is useful for queries involving phrases such as:

- Latest developments
- Recent research
- Current market conditions
- Events during the past year

However, the date alone does not make the agent's information current. The agent still needs suitable search tools, appropriate date filters when available, and a process for checking when its sources were published.

21.4 Defining Tools Clearly

Giving an agent access to a tool is not the same as explaining how to use it effectively.

A tool's technical definition may specify its name, parameters, and expected return type. These details are necessary, but the agent may also need instructions explaining the tool's purpose and role in the larger process.

Imagine your research agent has access to three tools:

Tool	Purpose
`delete_task`	Removes existing task entries from the tracker when appropriate
`append_update_task`	Creates a task or updates its stored information
`Search Worker Agent`	Executes a research search and returns findings

The exact tool names depend on the implementation. What matters is that the

agent understands the purpose of each tool and the conditions under which it should be used.

21.4.1 Example: Tool Usage Instructions

```
TOOL USAGE

Task management:
- Use the task management tools to maintain the research plan.
- Start each new research request with a clean task list, but
only
  clear entries belonging to the current research session.
- Create the task entries before delegating searches.
- Use only the permitted task status values.
- Update the relevant task after processing the worker's
response.

Search execution:
- Delegate search tasks to the Search Worker Agent.
- Provide the actual search query and any required search
constraints.
- Review the returned findings before recording the task as
completed.
- Do not mark a search as successful when the worker reports a
failure.

Final report:
- Verify task statuses before writing the report.
- Identify important information gaps.
- Do not present unverified findings as established facts.
```

These instructions explain how tool usage fits into the overall process.

For example, the agent should not mark a search as completed simply because it called the worker. It should examine the response and determine whether the task was actually completed.

21.4.2 Why Explicit Status Values Matter

Suppose the task tracker accepts these status values:

```
todo
in_progress
done
failed
```

Without an explicit definition, the agent might use `pending`, `completed`, `finished`, or `success` interchangeably.

That can create problems when another component expects a specific value.

Define what each allowed value means:

Status	Meaning
`todo`	The task has been created but has not started.
`in_progress`	The task is currently being executed.
`done`	The task was completed successfully and its result was recorded.
`failed`	The task could not be completed after the permitted recovery attempts.

If your system uses these values, instruct the agent to use them exactly as written.

A task should not be marked `done` if the search failed or its result has not been recorded.

A practical lesson: When an agent interacts with software, important expectations should be reflected in both the instructions and the actual tool or data design.

Where possible, enforce valid status values, required fields, and access permissions through the implementation itself instead of relying entirely on the model to follow instructions.

21.5 Choosing Between Flexible and Strict Instructions

One important design decision is how much freedom the agent should have when carrying out a research plan.

Consider two different approaches.

21.5.1 Approach A: Flexible Execution

A flexible agent can consolidate redundant tasks, adjust its plan, or decide that a planned search is unnecessary.

For example:

```
Execute the research plan efficiently.

If two search tasks substantially overlap, you may consolidate
them
before execution and record the reason.
```

```
If new information makes a planned task unnecessary, explain
why
and update the task tracker accordingly.

Do not silently skip tasks or leave task statuses unresolved.
```

This approach allows the agent to adapt its process to the research question.

It can reduce unnecessary searches, avoid duplicate work, and make more efficient use of resources.

However, the agent must still account for every task and demonstrate that the research objectives have been addressed.

21.5.2 Approach B: Strict Execution

A strict agent must execute every task in the approved plan.

For example:

```
STRICT EXECUTION RULES

- Execute a search for every approved search task.
- Do not skip or consolidate tasks during execution.
- Record the search result or failure for every task.
- Do not mark a task as done unless its search was completed
  successfully and the result was recorded.
- Verify that every approved task has reached a final status
  before preparing the report.
```

This approach is appropriate when completeness is more important than avoiding duplicate work.

For example, a research procedure may require independent searches to examine several specified topics separately, even when some findings overlap.

21.5.3 Which Approach Should You Choose?

Neither approach is universally better.

Consideration	Flexible Execution	Strict Execution
Adaptability	Higher	Lower
Task completeness	Requires careful validation	Easier to enforce for approved tasks

Continued on next page

Consideration	Flexible Execution	Strict Execution
Efficiency	Can avoid redundant work	May perform unnecessary searches
Control	More decisions delegated to the agent	Execution requirements are more explicit
Best suited for	Exploratory research	Fixed research procedures

You should also avoid giving contradictory instructions.

For example, telling the agent to execute exactly five searches while also allowing it to skip any search it considers unnecessary creates ambiguity about the actual requirement.

Decide whether the number of searches is mandatory, a planning guideline, or a minimum coverage target. Then express that decision consistently throughout the prompt and workflow.

A useful rule: Be strict about the outcomes and constraints that genuinely matter, while allowing flexibility in areas where different approaches can achieve the same result.

21.6 Improving Research Task Planning

A research agent's results depend heavily on the quality of its search plan.

A broad request such as "Research AI agents" is not a particularly useful search task. It could refer to architecture, planning, memory, tool use, evaluation, applications, or several other subjects.

The orchestrator should break the request into distinct information needs before delegating searches.

21.6.1 Example: Weak vs. Better Task Decomposition

Weak plan:

```
Task 1: Research AI agents.
Task 2: Search for AI agents.
Task 3: Find information about AI agents.
```

These tasks overlap heavily and do not establish clear research objectives.

Better plan:

```
Task 1: Identify the main architectural patterns used to build
AI agents and explain how they differ.
```

```
Task 2: Investigate common reliability challenges in agentic
systems,
including tool execution, planning, and context management.

Task 3: Find recent evaluations or case studies that report how
agentic systems perform on complex tasks.
```

The improved plan assigns each task a distinct purpose.

The exact searches will depend on the user's request, but the tasks should collectively cover the most important questions.

21.6.2 Add Metadata to Search Tasks

As your research system becomes more sophisticated, a search query alone may not contain enough information to guide execution.

You can extend each task with useful metadata.

For example:

Field	Purpose	Example
`task_id`	Identifies the task	`task_001`
`query`	Contains the actual search request	`AI agent evaluation methods`
`search_type`	Suggests the appropriate source category	`academic`
`date_range`	Specifies a requested time period	`past_year`
`domain_focus`	Identifies the research subject	`AI research`
`priority`	Helps determine execution order	`high`
`status`	Tracks task progress	`todo`

Not every field is necessary for every project.

For example, a general research assistant may not need a priority field for a small query. A system that performs academic research or monitors recent developments may benefit from source types and date filters.

The important point is that task metadata should support actual execution. Adding fields that the agent and tools do not use only creates unnecessary complexity.

21.6.3 Specify Date Ranges Carefully

Temporal instructions are particularly important for research.

Consider these requests:

- Find today's announcements.
- Review developments from the past month.
- Compare research published during the past year.
- Investigate how the industry changed between 2020 and 2024.

They require different interpretations of time.

Your system should define how time expressions are converted into search constraints and how explicit dates are represented.

For example:

```
DATE RANGE RULES

- Use the current date as the reference point for relative
periods.
- Interpret "past month" as the preceding calendar-length month
  unless the application defines a different convention.
- Represent explicit start and end dates in YYYY-MM-DD format.
- Do not invent date ranges when the user's intent is unclear.
- If the selected search tool does not support the requested
  date constraints, explain the limitation or use an
  appropriate
  alternative when available.
```

The exact rules should match your application's requirements. The main objective is to make the interpretation consistent and testable.

21.7 Designing Communication Between Agents

When an agent delegates work to another agent, you must determine what information the worker needs and what it should return.

Sending the entire conversation history to every sub-agent may seem convenient, but it can increase context length and introduce irrelevant details.

A better design is to provide each worker with the smallest amount of information needed to complete its task reliably.

21.7.1 What Should the Search Worker Receive?

For a basic search task, the worker might need:

- The search query

- The purpose of the search
- Any required date or source restrictions
- Instructions for handling failures
- The expected output format

It generally does not need the orchestrator's entire conversation history, unrelated search results, or the complete research report.

For example:

```
SEARCH TASK

Query:
Recent studies evaluating the reliability of AI agents.

Objective:
Identify relevant studies and summarize their findings.

Requirements:
- Prefer relevant primary sources when available.
- Record publication dates and source references.
- Distinguish reported findings from your own interpretation.
- Report search failures or limitations explicitly.
```

This gives the worker enough context to perform the task without transferring unnecessary information.

21.7.2 What Should the Worker Return?

The orchestrator needs more than an unstructured paragraph if it must update task statuses and synthesize multiple results.

Define a consistent response format.

For example:

```
{
  "status": "done",
  "query": "Recent studies evaluating the reliability of AI
  agents",
  "findings": [
    {
      "summary": "Summary of a relevant finding",
      "source": "https://example.com/research",
      "published_date": "2026-04-15"
    }
  ],
  "limitations": [],
```

```
  "error": null
}
```

This is an illustrative response structure, not a claim about any particular tool's actual output.

In a real implementation, validate the output against a schema. For example, `status` should accept only permitted values, and the orchestration system should verify that required fields are present.

A worker should never invent a source URL or publication date to satisfy the format. When information is unavailable, the response should represent that uncertainty explicitly.

21.7.3 Why Standardized Communication Helps

Consistent inputs and outputs make it easier for the orchestrator to:

- Update the correct task.
- Distinguish successful searches from failures.
- Combine findings from multiple workers.
- Identify missing information.
- Debug unexpected results.

The principle is simple: Each agent should receive the context it needs and return information that the next component can reliably use.

21.8 Managing Context Length

During a long research process, an agent's context can grow substantially.

Every search may add new findings, source details, task updates, and messages from other agents. Eventually, the accumulated information can become unnecessarily large.

This creates several potential problems:

- Important instructions may become harder to locate among irrelevant details.
- Repeated information consumes context that could be used for new findings.
- Large worker responses can make orchestration more expensive.
- Previous tasks may remain in context even after their details are no longer useful.

Here are four practical strategies.

21.8.1 Strategy 1: Separate Responsibilities

Using dedicated workers can prevent every agent from accumulating the complete context of the entire system.

The search worker handles its assigned search. The orchestrator maintains the broader research plan.

Use this separation when the improvement in focus and context management justifies the additional orchestration complexity.

21.8.2 Strategy 2: Summarize Results

Instead of carrying complete search-result payloads through every subsequent step, extract the most relevant findings and preserve their source references.

For example, a worker might return:

```
Finding:
Several studies identify tool reliability as an important
challenge
for multi-step AI agents.

Evidence:
- Source A: Relevant evaluation results.
- Source B: Discussion of tool-use failures.

Limitations:
The available sources do not establish that the same failure
rate
applies across all agent architectures.
```

A concise summary preserves the important information while avoiding unnecessary detail.

However, summarization must not remove evidence needed for later verification. Keep links, identifying details, and relevant quotations or data when they are necessary to support the final report.

21.8.3 Strategy 3: Track State Externally

Use a spreadsheet, database, or other persistent store to record task status and important findings.

The agent can retrieve the relevant state when needed rather than relying exclusively on the full conversation history.

External storage also makes it easier to inspect execution after a failure.

21.8.4 Strategy 4: Reset Context at Appropriate Boundaries

If the system handles multiple independent research requests, distinguish information belonging to one request from information belonging to another.

For example, a new research session should not accidentally inherit unfinished tasks from a previous user's investigation.

Clear or isolate session-specific state at appropriate boundaries, while preserving any longer-term memory that the application genuinely needs.

Be careful when deleting task records: remove only the entries belonging to the intended session, not unrelated work.

21.9 Adding Error Handling

Even a carefully designed search agent will encounter failures.

A source may be unavailable, an API may return an error, or a worker may produce an incomplete response.

Your instructions should explain how the system should respond instead of assuming that every operation will succeed.

For example:

```
ERROR HANDLING

- If the Search Worker Agent fails, retry once with a revised
query
  when an alternative query is likely to help.
- If the retry fails, record the failure and its reason.
- Mark an unsuccessful task as "failed", using the permitted
status.
- Continue with independent tasks when doing so remains useful.
- Do not report a failed search as successfully completed.
- If a critical failure prevents a reliable report, notify the
user
  and explain what remains incomplete.
- Before submitting the report, check whether unresolved
failures
  materially affect its conclusions.
```

These instructions establish a recovery strategy without requiring the entire process to stop whenever a single search fails.

However, retrying is not always appropriate. A permission error, invalid credential, or unavailable service may require intervention rather than another identical request.

The system should also distinguish between a tool call that failed and a search that succeeded but found no relevant information. Those are different out-

comes and should be recorded accordingly.

For reliable implementations, combine prompt-level instructions with programmatic error handling, retry limits, timeouts, and validation.

21.10 The Context Engineering Improvement Cycle

The architecture and prompt we've designed are starting points, not final solutions.

You need to test the agent and refine it according to its observed behavior.

A practical development cycle looks like this:

Step 1: Build the initial system.

Create the orchestrator, search worker, available tools, and task tracker. Begin with clear but relatively simple instructions.

Step 2: Test different research questions.

Use a range of inputs, from narrow factual questions to broad research topics.

Include cases where sources are unavailable, tasks overlap, or search results are incomplete.

Step 3: Observe the execution.

Examine task creation, tool calls, returned results, task status changes, and the final report.

Step 4: Identify the underlying issue.

Determine whether the failure came from unclear instructions, poor task decomposition, tool problems, inconsistent data, or excessive context.

Step 5: Make a targeted improvement.

Change the relevant prompt, tool definition, data schema, or architectural component.

Avoid changing several unrelated elements at once when doing so would make the results difficult to interpret.

Step 6: Run the tests again.

Compare the updated agent's behavior with its previous performance. Check whether the original problem has been resolved and whether new problems have appeared.

Step 7: Repeat.

Continue improving the system as you discover new failure patterns.

21.10.1 What Should You Measure?

You can track several indicators to assess whether the agent is becoming more reliable.

Metric	What to Check
Task completion	Were all required research tasks completed or explicitly accounted for?
Status correctness	Do task statuses accurately represent what happened?
Search quality	Were the searches relevant to the assigned questions?
Report completeness	Does the final report address the important research questions?
Error handling	Were failures recorded and handled appropriately?
Efficiency	How many tool calls, retries, tokens, and model operations were required?

A system that completes every task but generates a poor report is not successful. Likewise, an agent that produces a good report by silently skipping required operations may violate the intended process.

Evaluate both the final result and the execution that produced it.

21.11 Try It Yourself

Task: Imagine you've built a deep research agent with a single orchestrator that plans searches, executes them, manages a spreadsheet, and writes the final report.

During testing, you discover four problems:

- The agent creates five tasks but executes only three.
- It sometimes uses inconsistent task statuses such as `completed`, `done`, and `finished`.
- Its context grows so large that it begins repeating searches and overlooking previous findings.
- When a search fails, the final report does not mention the missing information.

Design an improved system.

Your solution should address:

1. Whether you would keep one agent or introduce a dedicated search worker.
2. Which responsibilities belong to the orchestrator and which belong to the worker.
3. What instructions and task status values you would define.
4. What information the worker should receive and return.
5. How you would manage context length and handle failures.
6. How you would test whether the revised system is more reliable.

21.11.1 Example Answer

1. Improve the architecture

I would introduce a dedicated search worker if the orchestrator's combined responsibilities were contributing to missed tasks or excessive context.

The orchestrator would plan the research, maintain task state, evaluate findings, and prepare the report. The worker would execute an assigned search and return structured findings.

For a small research application, I would first consider whether clearer instructions and better state tracking could solve the problems without adding a second agent.

2. Define responsibilities

The orchestrator would create the research plan, record tasks before execution, delegate searches, process responses, update task statuses, and verify completion.

The worker would receive a search query, execute the search using its available tools, summarize relevant findings, and report any failures.

3. Standardize instructions and statuses

I would define a fixed set of permitted status values:

- `todo`: The task has not started.
- `in_progress`: The task is being executed.
- `done`: The task completed successfully and its results were recorded.
- `failed`: The task could not be completed after the permitted recovery attempts.

Before submitting the report, the orchestrator would verify that every important task has an appropriate final status.

4. Standardize communication

The worker would receive only the search query, relevant constraints, and instructions needed for the assigned task.

It would return a structured response containing its status, findings, source references, and any limitations or errors.

The system would validate the response before updating the tracker.

5. Manage context and errors

I would store task state outside the conversation, summarize findings while preserving their source references, and avoid repeatedly passing irrelevant history to the worker.

If a search failed, the system would retry once with a revised query when appropriate. If it failed again, the tracker would record the failure, and the orchestrator would decide whether the remaining research could continue reliably.

The final report would explicitly identify any important unresolved questions.

6. Evaluate the changes

I would test the system using research questions that include overlapping tasks, unavailable sources, incomplete results, and different levels of complexity.

I would compare task completion, status correctness, search quality, report completeness, and execution cost before and after the changes.

The goal would be to improve reliability while keeping the architecture and context no more complicated than necessary.

21.12 What's Next

You've now explored how context engineering applies to a real agent architecture, from separating responsibilities and defining tools to managing task state, controlling context growth, and handling failures.

The key takeaway is that **reliable AI agents emerge from the combination of clear context, focused responsibilities, well-defined interfaces, and systematic testing**. A longer system prompt alone cannot compensate for an architecture that makes important operations difficult to track or validate.

As you build more advanced agentic systems, these principles will help you decide where to introduce additional components, which information to preserve, and how to evaluate whether your design is actually improving performance.

22 Function Calling in AI Agents

You've learned how context engineering helps AI agents use instructions, tools, and memory more effectively. You've also explored how separating responsibilities can make complex agentic systems easier to manage and debug.

But how does an AI agent actually interact with an external tool?

Imagine asking an AI assistant, "What is the weather in Paris right now?"

A language model can understand your question, but it cannot retrieve live weather conditions from its training data alone. It needs access to an external service that provides current weather information.

The model must identify the appropriate tool, provide the required information, receive the result, and use that result to answer your question.

This is where **function calling** comes in.

In this guide, you'll learn how function calling works, how tool definitions influence an agent's behavior, how actions and observations form an agent loop, and how to debug common problems.

22.1 What Is Function Calling?

Function calling, also known as **tool calling**, is a capability that allows a language model to request the execution of predefined functions or tools.

These tools can connect the model to external resources such as:

- APIs that provide current information.
- Databases that store application data.
- Search engines that retrieve relevant documents.
- Calculators that perform precise computations.
- Software applications that perform actions on behalf of a user.

The model does not need to implement these capabilities itself. Instead, it can select an available tool and produce a structured request describing how that tool should be used.

However, there is an important distinction:

Function calling does not automatically execute the function. It allows the model to request a function call, which the surrounding application can then execute.

Consider the weather example.

The user asks:

> What is the weather in Paris right now?

The process might look like this:

1. The model identifies that the request requires current weather information.
2. It selects a tool named `get_current_weather`.
3. It generates the required arguments, specifying Paris as the location.
4. The application receives the tool call and executes the corresponding function.
5. The weather service returns the current conditions.
6. The application passes the result back to the model.
7. The model uses the result to formulate a response.

The final answer might be:

> It is currently 18°C in Paris, with cloudy skies.

The temperature and conditions in this example are illustrative, not live weather data.

Without function calling, the model would need to rely on information already available in its context. With function calling, it can request information from an external source and incorporate the returned result into its response.

22.2 How Function Calling Powers AI Agents

Function calling connects a model's ability to interpret requests with the actions that an application can perform.

To understand how this works, let's examine the complete process.

22.2.1 Step 1: The User Sends a Request

The process begins with a user asking a question or requesting an action.

For example:

```
User: What is the weather in Paris?
```

The application receives the request and prepares the information that will be sent to the model.

22.2.2 Step 2: The Application Assembles the Context

The model needs information about the conversation and the tools available to it.

Depending on the implementation, the context may include:

- System instructions defining the model's role.
- The user's current request.
- Relevant conversation history.
- Tool definitions describing available functions.
- Other task-specific information.

For example, the application might tell the model that it has access to a `get_current_weat` function and explain what information that function requires.

These definitions help the model determine which tool is appropriate for the request.

The application does not necessarily include the entire history of every interaction. It provides the context available for that particular model call, which may be assembled, summarized, or limited by the implementation.

22.2.3 Step 3: The Model Selects a Tool

The model evaluates the request and determines whether a tool is needed.

If it decides that a function call is appropriate, it returns a structured tool request.

For example:

```
{
  "name": "get_current_weather",
  "arguments": {
    "location": "Paris",
    "unit": "celsius"
  }
}
```

This structure identifies the function and provides its arguments.

At this point, the model has requested a function call. The weather service has not necessarily been contacted yet.

The application must still receive and execute the request.

22.2.4 Step 4: The Application Executes the Function

The application interprets the model's request and invokes the corresponding implementation.

For example, the application might call a weather API with the location `Paris` and request the temperature in Celsius.

The actual implementation could use an HTTP request, a software library, or another mechanism. The model does not need to know the internal details as

long as the tool interface is clearly defined.

The application should also validate the requested function and its arguments before execution, particularly when the tool can modify data or perform consequential actions.

22.2.5 Step 5: The Tool Returns an Observation

After execution, the tool returns its result.

For example:

```
{
  "location": "Paris",
  "temperature": 18,
  "unit": "celsius",
  "conditions": "Cloudy"
}
```

In agent terminology, this returned information is often called an **observation**.

An observation tells the agent what happened after an action was attempted.

A successful search might return relevant documents. A calculator might return a numerical result. A database query might return a collection of records.

A tool can also return an error or indicate that no results were found.

22.2.6 Step 6: The Result Returns to the Model

The application adds the tool result to the conversation or execution context and sends it back to the model.

The model can then interpret the result and decide what to do next.

For the weather example, it may now have enough information to answer the user.

For a more complicated task, it might need another tool call.

22.2.7 Step 7: The Model Produces a Response

Once the model has sufficient information, it can generate a final response based on the user's request and the available observations.

For example:

```
The current weather information for Paris reports
a temperature of 18°C with cloudy conditions.
```

This response illustrates how the tool result could be communicated to the user. A production system should preserve appropriate timestamps and distinguish current observations from stale or unavailable data.

22.2.8 The Complete Flow

The entire process can be summarized as:

User Request Context Assembly Tool Selection Tool Execution Observation Model Response

If the task requires additional operations, the process can repeat.

For example, an agent researching a company might search the web, inspect a promising result, retrieve another source, compare findings, and then produce a summary.

The important point is that the model and its tools play different roles: the model requests and interprets actions, while the application executes the corresponding functions.

22.3 The Role of Tool Definitions

An agent cannot reliably select an appropriate tool unless it has enough information about the available options.

This is why **tool definitions** are an important part of function calling.

A tool definition describes a function's identity, purpose, and expected inputs. Depending on the interface, it may also describe output constraints or other metadata.

A typical tool definition contains three essential elements:

Element	Purpose
Name	Identifies the function the model can request.
Description	Explains what the function does and when it should be used.
Parameters	Defines the inputs the function expects, including their types and constraints.

Let's examine each element.

22.3.1 The Name

A function name should describe its purpose clearly.

For example:

```
get_current_weather
```

This is more informative than a vague name such as:

```
execute_function
```

Clear naming helps distinguish tools when an agent has several functions available.

22.3.2 The Description

The description explains what a function does and when it is appropriate to use it.

Consider these two descriptions.

Weak description:

```
Get weather.
```

Better description:

```
Retrieve current weather conditions for a specified location.
Use this tool when the user asks about present weather
conditions
and live weather information is required.
```

The second description provides useful context for tool selection.

This becomes especially important when multiple tools have similar purposes. For example, an agent might have access to both a current-weather tool and a historical-weather tool.

Their descriptions should make the difference clear.

22.3.3 The Parameters

Parameters define the information a function needs to execute.

For example, a weather tool may require a location and optionally accept a temperature unit.

A parameter definition can specify:

- The parameter's name.
- Its data type.
- A description of what it represents.
- Whether it is required.

- Permitted values, where applicable.

Constraints help reduce ambiguity and make the tool interface more predictable.

22.3.4 A Complete Tool Definition

Here is an illustrative function definition using a JSON-style schema and Python syntax:

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_weather",
            "description": (
                "Retrieve current weather conditions for a
                location. "
                "Use this when the user asks about present
                weather "
                "conditions and live information is required."
            ),
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": (
                            "The city or region to retrieve
                            weather for, "
                            "such as Paris, France."
                        )
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": (
                            "Temperature unit for the returned
                            result."
                        )
                    }
                },
                "required": ["location"]
            }
        }
    }
]
```

This example uses a common function-definition format. Exact syntax and supported schema features can vary across model providers and APIs.

Notice that the tool definition describes the interface, not the implementation of the weather service itself.

The actual function that contacts the weather API must exist in the application.

22.3.5 Why Tool Definitions Matter So Much

Suppose an agent has access to these tools:

```
search_web(query)
search_internal_documents(query)
get_customer_order(order_id)
update_customer_order(order_id, status)
```

If the descriptions are vague, the model may select the wrong tool or provide inappropriate arguments.

For example, a question about an internal company policy should usually be directed to the internal document search tool rather than a general web search.

You can reduce these mistakes by clearly explaining each tool's purpose and the conditions under which it should be used.

There is also a practical trade-off: tool definitions become part of the information supplied to the model. Providing many lengthy definitions can increase context size, token usage, cost, and latency.

Aim for tool definitions that are concise but specific enough for the model to select and use each tool correctly.

22.4 Understanding the Agent Loop: Actions and Observations

Function calling becomes particularly powerful when the model can use the result of one action to decide what to do next.

This repeated process is commonly called the **agent loop**.

At a high level, the loop contains four stages:

1. **Action:** The model requests an operation using an available tool.
2. **Environment response:** The application executes the operation and receives a result.
3. **Observation:** The result is made available to the model.
4. **Decision:** The model determines whether another action is necessary or whether it can respond to the user.

Let's examine a practical example.

22.4.1 Example: Researching Recent Information

Suppose a user asks:

```
Find recent announcements from OpenAI and summarize the main
updates.
```

The agent has access to a `web_search` tool.

Its first action might be:

```
Action:
web_search(
    query="OpenAI recent official announcements"
)
```

The search tool returns several results.

```
Observation:
- An official announcement about a new product.
- A company update describing a new feature.
- A separate announcement about an API capability.
```

The agent now examines these observations.

It might determine that one result provides enough information for a simple answer. Alternatively, it may decide that additional searches are necessary to verify dates or investigate another announcement.

The process could continue:

```
Action:
web_search(
    query="site:openai.com/index/ official API announcement"
)
```

The agent receives another observation and evaluates whether it has enough information to answer.

Once the relevant information has been collected, it synthesizes the findings into a response.

The example illustrates the interaction between the model and the tool. Actual search results depend on the query, available search service, and time of execution.

22.4.2 Why the Loop Matters

A tool call is not always the end of a task.

The result of one operation can determine which operation should happen next.

For example, a customer support agent might:

1. Retrieve an order using its order ID.
2. Examine the delivery status.
3. Request tracking information if the order is delayed.
4. Determine whether the available evidence explains the delay.
5. Provide an answer or route the unresolved issue to a support representative.

The exact sequence depends on the information returned by the tools.

This is one of the key differences between a single tool call and an agent that uses tools iteratively.

22.4.3 Does the Model Always Keep the Entire History?

Not necessarily.

For the agent to make decisions based on previous actions, the application must provide the relevant information from those actions.

Many implementations keep prior messages and tool results in the current context. Others may summarize earlier results, retrieve information from memory, or omit irrelevant history when preparing subsequent model calls.

The essential requirement is that the agent receives enough relevant context to make its next decision.

If important observations are omitted, the agent may repeat work, misunderstand the current state, or make a decision that conflicts with an earlier result.

22.5 Debugging Function Calls

Even when the overall design is sound, function calling can fail in several ways.

Perhaps the model chooses the wrong tool, supplies an invalid argument, or fails to recognize that external information is required.

Understanding the execution process makes these problems easier to investigate.

22.5.1 Common Function Calling Problems

Incorrect tool selection

The model chooses an unsuitable tool for the task.

For example, it uses a web search instead of the internal knowledge base when answering a question about company-specific documentation.

Incorrect arguments

The model selects the correct tool but supplies incomplete or invalid parameters.

For example, it calls `get_customer_order` without an order ID.

Insufficient tool instructions

The available tools have vague descriptions, so the model cannot reliably distinguish when each tool should be used.

Unexpected tool results

The function returns an error, empty data, or a response that differs from what the agent expected.

Incorrect observation handling

The model receives the tool result but misinterprets it or ignores important information.

Unnecessary tool calls

The agent calls tools even when the requested task can be completed using the existing context, increasing latency and cost.

Each problem requires a different solution. Adding more instructions to the system prompt may help, but the underlying issue could also be a tool schema, implementation bug, validation failure, or incomplete execution history.

22.5.2 Inspect Intermediate Steps

One of the most useful debugging techniques is to inspect the sequence of tool calls and results.

Some agent frameworks and workflow automation platforms provide intermediate execution logs or tracing features. Their exact names and capabilities depend on the platform.

The information worth examining includes:

- Which tool the model selected.
- The arguments passed to that tool.
- The tool's returned result.
- Any errors encountered.
- Whether the result was passed back to the model.
- What happened next in the execution flow.
- Token usage and execution time, when available.

For example, an execution trace might resemble the following:

```
{
  "intermediateSteps": [
    {
      "action": {
        "tool": "web_search",
        "toolInput": {
          "query": "OpenAI official recent announcements"
        }
      },
      "observation": {
        "status": "success",
        "result_count": 3
      }
    },
    {
      "action": {
        "tool": "update_task_status",
        "toolInput": {
          "taskId": "search_1",
          "status": "done"
        }
      },
      "observation": {
        "status": "success",
        "message": "Task updated"
      }
    }
  ]
}
```

This is an illustrative trace structure, not the exact output format of a particular framework.

The first entry records a search operation and its result. The second records an update to the task tracker.

By examining the sequence, you can determine whether the agent selected the appropriate functions, whether they executed successfully, and whether their results were processed correctly.

22.5.3 Trace the Failure to Its Source

Suppose the final report omits an important finding.

Rather than immediately changing the prompt, inspect the execution trace.

Did the agent create the relevant search task? Did it call the search tool? Did the search return useful information? Was the result passed back to the model? Was the task marked as completed correctly?

These questions help distinguish between a planning problem, a function calling problem, a tool implementation problem, and a result synthesis problem.

The goal of debugging is to find the stage where the expected behavior diverged from the actual behavior.

When possible, inspect the underlying model requests, tool responses, and execution logs rather than relying exclusively on a platform's simplified interface.

22.6 Best Practices for Function Calling

Reliable function calling requires good tool definitions, appropriate execution controls, and clear instructions.

Here are several practices to apply when building AI agents.

22.6.1 Practice 1: Write Specific Tool Descriptions

Tool descriptions should communicate both what the function does and when the agent should use it.

Weak:

```
Search the web.
```

Better:

```
Search the web for information from external sources.

Use this tool when the user asks about recent events, current
information, or facts that may have changed since the model's
training data.

Provide a focused query describing the information needed.
```

A specific description makes the tool easier to distinguish from alternatives.

22.6.2 Practice 2: Explain Tool Usage in System Instructions

Tool definitions explain individual functions, but the system prompt can describe how those functions should work together.

For example:

```
AVAILABLE TOOLS

- web_search:
```

```
  Use for current information from external sources.

- calculator:
  Use for arithmetic and numerical calculations that require
  precise results.

- internal_knowledge_base:
  Use for questions about internal company documentation.

TOOL SELECTION RULES

- Prefer the internal knowledge base for company-specific
questions.
- Use web_search when current external information is needed.
- Use calculator for calculations that should not rely on
  language-model estimation.
- If a tool fails, evaluate whether a retry or an alternative
  approach is appropriate.
```

These instructions establish a basic tool-selection policy.

However, they should remain consistent with the actual capabilities available to the agent. A tool that is not registered or accessible cannot be used simply because the system prompt mentions it.

22.6.3 Practice 3: Define Clear Parameter Constraints

Whenever possible, constrain tool parameters using appropriate types, required fields, enumerated values, and schema validation.

For example:

```
{
  "unit": {
    "type": "string",
    "enum": ["celsius", "fahrenheit"],
    "description": "The temperature unit for the result."
  }
}
```

This restricts the permitted temperature units to two values.

Without such constraints, the model might produce values such as `"degrees"` or `"kelvin"` even if the underlying function supports only Celsius and Fahrenheit.

Schema constraints do not guarantee that every call will be valid in every implementation, so applications should validate arguments before executing the

function.

22.6.4 Practice 4: Handle Tool Failures Gracefully

Tools should return meaningful results when something goes wrong.

Consider a database search that returns no matching records.

Instead of returning an empty response with no explanation, the function could return a structured result:

```
{
  "status": "success",
  "results": [],
  "message": "No matching records were found."
}
```

This tells the agent that the search executed successfully but did not find a match.

A different response should represent an actual execution failure:

```
{
  "status": "error",
  "results": [],
  "message": "The database service is temporarily unavailable."
}
```

These outcomes should not be treated as equivalent.

An empty result may mean the requested information does not exist or that the query was too narrow. A tool error means the system could not reliably complete the operation.

Meaningful error messages help the agent decide whether to adjust its query, retry, use an alternative tool, or explain the limitation to the user.

22.6.5 Practice 5: Validate Before Executing Actions

Function calling can retrieve information, but it can also trigger actions that affect external systems.

For example, a tool might send an email, update a customer's order, or delete a database record.

Before executing such operations, the application should verify that the requested action is permitted and that the arguments are valid.

For consequential or irreversible actions, additional safeguards may be necessary, such as explicit user confirmation or human approval.

Do not rely solely on the model's decision to determine whether an operation is safe to execute.

22.6.6 Practice 6: Keep Tool Definitions Focused

A large collection of overlapping tools can make tool selection more difficult and increase the amount of context supplied to the model.

Prefer clear interfaces with distinct responsibilities.

For example, a customer support system might separate:

```
get_order_status(order_id)
cancel_order(order_id)
update_delivery_address(order_id, address)
```

Each tool has a distinct purpose and a clear expected input.

The application should enforce authorization and business rules for every operation, regardless of which tool the model requests.

22.6.7 Practice 7: Monitor Tool Usage

For an agent operating in a real application, track relevant execution metrics.

Useful measurements include:

- Tool selection accuracy.
- Invalid argument frequency.
- Tool execution success rate.
- Number of retries and repeated calls.
- Average response time.
- Token usage and execution cost.
- Frequency of unresolved requests.

These measurements help you identify patterns that are difficult to discover from final responses alone.

For example, frequent calls to an unsuitable tool may indicate unclear descriptions or an inadequate tool-selection policy. Repeated execution failures may instead point to an unstable API or a problem with the tool implementation.

Use the evidence to decide which component needs improvement.

22.7 Try It Yourself

Task: Imagine you're building an AI assistant for an online store.

The assistant has access to three functions:

```
search_products(query)
get_order_status(order_id)
cancel_order(order_id)
```

The user asks:

> Find wireless headphones under 100 DT and tell me whether my order 4821 has shipped.

Answer the following questions:

1. Which function or functions should the agent call?
2. What arguments should it provide to each function?
3. In what form should the tool results be returned to the model?
4. What should happen if the order lookup fails?
5. Should the agent call `cancel_order` as part of this request? Explain why.

22.7.1 Example Answer

1. Select the appropriate functions

The agent should call `search_products` to find wireless headphones matching the user's budget and `get_order_status` to retrieve information about order 4821.

The user has not requested an order cancellation, so `cancel_order` is unnecessary.

2. Provide suitable arguments

The product search could use a query such as:

```
{
  "query": "wireless headphones priced under 100 DT"
}
```

The order lookup could use:

```
{
  "order_id": "4821"
}
```

Whether the order ID should be represented as a string or a number depends on the actual function schema.

3. Return structured observations

Each function should return the requested information in a format the application can validate and pass back to the model.

For the product search, that might include product names, prices, and relevant identifiers.

For the order lookup, the response might include the order's current status and the timestamp of the information, if available.

4. Handle failures

If the order lookup fails, the application should return an informative error instead of inventing a shipping status.

The agent could retry if appropriate or tell the user that the order status could not be retrieved.

5. Avoid the cancellation function

The user has not asked to cancel the order. Calling `cancel_order` would introduce an unintended action.

The agent should use only the tools necessary to fulfill the request and follow the application's authorization and confirmation rules for actions that modify an order.

22.8 What's Next

You've now learned how function calling connects language models to external tools, how tool definitions guide function selection, and how actions and observations form an iterative agent loop.

The key takeaway is that **function calling is the interface between an LLM's decisions and the operations an application can actually execute**. Reliable agents need more than the ability to request tools: they also require well-designed tool definitions, validated arguments, meaningful observations, appropriate execution controls, and clear error handling.

Next, we'll explore **Agent Components** to examine how planning, tool utilization, and memory work together to support an agent's behavior.

23 Deep Agents

You've learned how function calling allows AI agents to interact with external tools, how context engineering improves their behavior, and how multiple agents can collaborate on complex tasks.

But what happens when a task becomes so large that a simple agent struggles to manage it?

Imagine asking an AI assistant to research an entire industry, compare competitors, analyze market trends, examine dozens of sources, and produce a comprehensive report. The agent must plan its work, keep track of progress, remember important findings, and verify its conclusions.

A basic agent that handles everything in one continuous interaction may lose track of important details, repeat previous work, or overlook unfinished tasks.

To handle these challenges, developers are exploring more sophisticated agent architectures known as **Deep Agents**.

In this guide, you'll learn what Deep Agents are, how they approach complex tasks, and how planning, specialized agents, persistent memory, context engineering, and verification work together to improve reliability.

23.1 What Are Deep Agents?

Deep Agents are AI agent systems designed to handle complex, multi-step tasks by combining structured planning, persistent context, specialized execution, and verification.

Unlike a simple agent that responds to a request with a single tool call or a short sequence of actions, a Deep Agent is designed to coordinate a larger body of work toward a long-term goal.

For example, a basic research agent might search for information about a company and summarize the results.

A Deep Agent could approach the same task by:

1. Breaking the research question into smaller tasks.
2. Creating a structured plan and tracking its progress.
3. Delegating searches to specialized sub-agents.
4. Saving findings and intermediate work outside the immediate conversation.
5. Investigating gaps or conflicting information.
6. Checking whether the research objectives have been met.
7. Combining the results into a coherent final report.

The difference is not simply the number of steps involved. It is how the system **organizes, coordinates, remembers, and verifies its work over an extended task**.

The term Deep Agents describes an emerging approach to agent design rather than one universally standardized architecture. Different implementations may combine these capabilities in different ways.

23.1.1 Shallow Agents vs. Deep Agents

For learning purposes, it helps to compare two broad approaches.

Aspect	Simpler Agent Systems	Deep Agent Systems
Planning	May handle a short sequence of actions	Maintain structured plans for larger tasks
Task management	Often relies on the immediate interaction	Tracks progress across multiple subtasks
Execution	May handle most work in one agent	Can delegate work to specialized sub-agents
Memory	Often relies heavily on the current context	Can store intermediate work in external memory
Context management	May accumulate information in one conversation	Can organize, retrieve, and summarize relevant context
Verification	May rely mainly on the final response	Can include dedicated checks and evaluation stages
Best suited for	Straightforward, bounded tasks	Complex tasks involving many dependent operations

This comparison describes common design tendencies, not strict categories. A simple architecture can still be reliable, and a complex architecture can still fail.

The objective is not to make every agent deep. It is to provide the structure required by tasks that cannot be handled reliably through a simpler approach.

23.2 Planning: Turning Large Goals into Manageable Tasks

One of the most important characteristics of a Deep Agent is its ability to maintain a structured plan.

When a task involves many operations, the agent needs more than a general instruction to complete the work. It needs to understand what must be accomplished, which steps are necessary, and what remains unfinished.

Consider this request:

> Research the market for cloud-based business software and prepare a report for small-business owners.

This request contains several distinct objectives. The agent may need to examine market trends, identify customer challenges, compare competing products, and summarize the opportunities.

Attempting to complete everything without a plan makes it easier to overlook important questions.

23.2.1 Create a Structured Plan

The agent could begin by creating a task list:

Task ID	Task	Status
1	Identify major market trends	`todo`
2	Investigate small-business needs	`todo`
3	Compare competing products	`todo`
4	Identify adoption barriers	`todo`
5	Synthesize the findings into a report	`todo`

As the work progresses, the agent updates the status of each task.

For example, after completing the first search, the agent records its findings and marks the corresponding task as completed.

The plan becomes a representation of the work that has been completed and the work that remains.

23.2.2 A Plan Should Evolve

A Deep Agent should not necessarily treat its initial plan as an unchangeable sequence.

Suppose the competitor research reveals that most products target large enterprises rather than small businesses. This finding may suggest that the agent should investigate affordable alternatives or products designed for smaller companies.

The agent can update its plan to address the newly discovered question.

Similarly, if a search fails, it can retry, use another source, or record that the information could not be obtained.

This ability to revise a plan is particularly useful for tasks where the correct next step depends on what the agent discovers.

23.2.3 Why Planning Matters

Structured planning helps an agent:

- Keep track of outstanding work.
- Avoid repeatedly performing completed tasks.
- Identify dependencies between subtasks.
- Recover from failures.
- Determine when the overall objective has been satisfied.

However, a task list alone does not guarantee successful execution. The system must also track task state correctly and verify that completed tasks actually produced the expected results.

The key idea: A plan turns a broad objective into manageable work that the agent can track, execute, and revise.

23.3 Orchestrator and Sub-Agent Architecture

As tasks become more complex, one agent may have too many responsibilities to manage efficiently.

Imagine a research agent that must plan searches, operate tools, organize files, summarize findings, check sources, track task statuses, and write a report.

Every responsibility introduces additional instructions and information. As the task progresses, the agent's context can grow substantially.

One approach is to divide the work among an **orchestrator** and several **specialized sub-agents**.

23.3.1 What Is an Orchestrator?

An **orchestrator** coordinates the overall task.

It understands the user's goal, creates the plan, determines which work should be delegated, tracks progress, and combines the results into a final response.

It does not necessarily perform every operation itself.

23.3.2 What Are Sub-Agents?

Sub-agents are agents assigned specific responsibilities within the larger task.

For example, a research system might include:

Agent	Responsibility
Research Orchestrator	Plans the investigation, delegates tasks, and coordinates execution

Continued on next page

Agent	Responsibility
Search Agent	Searches for relevant information and returns findings
Analysis Agent	Examines collected information and identifies patterns
Verification Agent	Checks claims, evidence, and task completion
Writing Agent	Organizes verified findings into a report

Each sub-agent receives the information needed for its assigned task and returns results to the orchestrator.

This separation allows different agents to use different instructions, tools, and models according to their responsibilities.

23.3.3 How Do They Work Together?

Consider a user who asks for a report comparing three competitors.

The process could follow this sequence:

1. The orchestrator identifies the information needed for each competitor.
2. It creates research tasks and assigns them to search agents.
3. The search agents collect information and return their findings.
4. The orchestrator combines the results and identifies missing information.
5. An analysis agent compares the competitors using the collected evidence.
6. A verification agent checks important claims and identifies unsupported conclusions.
7. The orchestrator integrates the verified results into the final report.

The orchestrator remains responsible for the overall objective, while the sub-agents focus on their individual tasks.

23.3.4 Why Separate Responsibilities?

This architecture offers several potential benefits.

Focused responsibilities

A search agent does not need to manage the entire research plan. It can concentrate on finding and summarizing relevant sources.

More manageable context

Each agent can work with a smaller, task-specific context instead of receiving the entire conversation history.

Flexible model selection

A planning task may require stronger reasoning capabilities, while a straightforward search or classification task may work well with a faster, less expensive model.

Independent evaluation

Different components can be tested separately. If search quality is poor, you can investigate the search agent without immediately redesigning the entire system.

23.3.5 Do You Always Need Multiple Agents?

No.

Multi-agent architectures introduce coordination overhead. The orchestrator must delegate tasks, process outputs, manage failures, and reconcile potentially conflicting findings.

A single agent may be simpler and more cost-effective for tasks that do not require specialized responsibilities.

Use sub-agents when dividing the work provides a meaningful benefit, such as reducing context complexity, enabling independent research, or supporting specialized execution.

The key idea: An orchestrator coordinates the overall goal, while specialized sub-agents handle focused parts of the work.

23.4 Context Retrieval and Persistent Memory

Long-running tasks create a memory challenge.

As an agent searches for information, it accumulates findings, intermediate results, tool outputs, and decisions. Keeping everything in one conversation can make the context increasingly large and difficult to manage.

Deep Agents can address this problem by storing important information outside the immediate conversation.

23.4.1 What Is Persistent Memory?

Persistent memory is information stored so that the system can retrieve and reuse it beyond the immediate model interaction.

Depending on the application, the stored information might include:

- Research notes and summaries.
- Task lists and execution status.
- Source references and extracted evidence.
- Intermediate analysis results.
- Files created during execution.
- Relevant information from previous interactions.

For example, a research agent might save a summary of the competitor analysis to a file rather than repeatedly including every original search result in its context.

Later, when the agent needs to compare pricing, it can retrieve the relevant findings and continue working.

23.4.2 Different Ways to Store Information

There are several ways to provide external memory.

Files and documents

Useful for research notes, reports, code, structured plans, and other intermediate work.

Databases

Useful for structured information such as task status, customer records, and execution history.

Vector databases

Useful for retrieving relevant passages or documents based on semantic similarity.

Conversation or task state

Useful for preserving the current objective, completed steps, pending work, and other information needed to resume execution.

These approaches solve different problems. A vector database is not a replacement for a structured task tracker, and a task tracker is not necessarily the best place to store large documents.

The appropriate storage depends on what information must be preserved and how it needs to be retrieved.

23.4.3 Context Retrieval

Storing information is only half of the problem. The agent must also retrieve the appropriate information when needed.

Suppose a research agent has collected information about ten competitors. When investigating one competitor's pricing, it should retrieve the relevant notes rather than loading every finding from the entire investigation.

The system can retrieve information using several approaches.

Keyword search finds information containing specific terms.

Semantic search finds information related to a query based on meaning rather than exact wording.

Structured lookup retrieves information using identifiers, filters, or predefined database queries.

A more advanced system can combine these approaches and select an appro-

priate retrieval strategy for the task.

For example, it might use a structured query to retrieve the current status of a research task, then use semantic search to locate relevant evidence in previously collected documents.

23.4.4 Memory Requires Careful Management

Persistent storage does not automatically make an agent more reliable.

Stored information can become outdated, incomplete, duplicated, or associated with the wrong task.

A good memory design should distinguish between current state, historical findings, verified facts, and unresolved assumptions.

It should also preserve relevant source references and update information when newer evidence changes an earlier conclusion.

The key idea: Persistent memory allows an agent to preserve important work, while context retrieval helps it access the information needed for the current step without carrying everything into every model interaction.

23.5 Context Engineering for Deep Agents

You've already learned that context engineering involves designing the information available to an AI agent.

For a Deep Agent, context engineering becomes especially important because the system may involve multiple agents, numerous tools, persistent memory, and long-running task plans.

The challenge is not simply writing a detailed system prompt. It is deciding which information each component needs, how instructions should be organized, and how context should change as the work progresses.

23.5.1 Make Instructions Explicit

The orchestrator should know when to create a plan, when to delegate work, and how to track results.

A search agent should know what information to collect, which tools it can use, and what its response must contain.

A verification agent should know what evidence to check and how to report unsupported claims.

For example, an orchestrator might follow these instructions:

```
ORCHESTRATION RULES

1. Break complex requests into manageable tasks.
2. Record tasks before delegating them.
```

```
3. Delegate specialized work to the appropriate sub-agent.
4. Provide each sub-agent with only the context needed for its
task.
5. Record and review each returned result.
6. Revisit the plan when new information changes the required
work.
7. Verify that important tasks are completed or explicitly
   identified as unresolved before returning the final result.
```

These instructions describe expectations that the orchestrator can apply across different research tasks.

23.5.2 Define How Agents Collaborate

When several agents work together, the system needs clear communication rules.

For example, each sub-agent might be required to return:

- Its task identifier.
- Its completion status.
- Its findings.
- Supporting sources or evidence.
- Any errors or limitations.
- Questions that require further investigation.

A consistent response format makes it easier for the orchestrator to integrate the results.

It is also important to define what happens when agents disagree. The orchestrator should not automatically accept whichever answer appears most confident. Conflicting claims may require additional evidence or independent verification.

23.5.3 Manage Context Deliberately

Deep Agents should not receive every available instruction and every stored result by default.

Instead, provide the context that is relevant to the current operation.

A search agent might need a query, a source restriction, and an expected output format. It usually does not need the complete research conversation.

The orchestrator, by contrast, needs enough information to understand the overall objective and integrate the results.

This approach keeps responsibilities distinct and makes context easier to maintain.

23.5.4 Keep Instructions Consistent

Context often comes from several places: system prompts, task-specific instructions, tool descriptions, retrieved documents, and stored state.

These sources should not contradict one another.

For example, the system prompt might require the agent to verify every source, while a task instruction tells it to produce the final report immediately after the first search.

Such conflicts can lead to unpredictable behavior.

Review the entire context design, not just the main prompt, when investigating inconsistent results.

The key idea: Context engineering makes sure each component has the relevant instructions and information needed to perform its responsibilities effectively.

23.6 Verification: Checking the Agent's Work

A Deep Agent may perform many operations and produce a detailed result, but complexity does not guarantee correctness.

The system might retrieve inaccurate information, misinterpret a source, overlook a required task, or combine findings into an unsupported conclusion.

This is why **verification** is an important part of the architecture.

Verification involves checking whether the agent's outputs and actions satisfy the required standards.

23.6.1 What Should Be Verified?

Different tasks require different forms of verification.

For a research agent, you might check:

- Whether important claims have supporting sources.
- Whether the sources actually support those claims.
- Whether publication dates and other relevant details are accurate.
- Whether conflicting findings have been acknowledged.
- Whether all required research questions have been addressed.
- Whether the final report follows the requested format.

For a coding agent, verification might involve running tests, checking program behavior, and examining whether the implementation satisfies the requirements.

For a customer support agent, it might involve checking the relevant account records and confirming that the proposed action follows the applicable policy.

Verification should be tailored to the task rather than treated as a generic instruction to ”check your work.”

23.6.2 Automated Verification

A system can use tools or specialized evaluation components to check outputs.

For example, a research report verifier might examine whether each major factual claim includes a source reference and whether the references can be retrieved.

An LLM-based evaluator can also assess qualities such as relevance, completeness, and consistency against an explicit rubric.

This approach is sometimes called **LLM-as-a-Judge**.

However, a language model evaluating another language model is not an infallible source of truth. The evaluator may repeat similar mistakes or approve a convincing but incorrect claim.

Whenever possible, combine model-based evaluation with external evidence, deterministic checks, or tests.

23.6.3 Human Verification

Some decisions require human judgment or authorization.

For example, an agent might prepare a financial analysis, but an analyst should review the result before it is used to make a consequential decision.

Likewise, an agent that proposes a significant account change may need explicit approval before executing it.

Human review is particularly useful when errors have substantial consequences, the evidence is ambiguous, or the system operates outside its validated conditions.

23.6.4 Verification Should Influence What Happens Next

Verification is more useful when its results can guide further work.

Suppose a verification agent discovers that two important claims in a research report lack reliable evidence.

Instead of simply labeling the report as incorrect, the system can send those claims back for further investigation.

The orchestrator may create additional search tasks, retrieve alternative sources, and update the report before running verification again.

The process becomes:

1. Produce an initial result.
2. Verify the result against defined requirements.
3. Identify failures or missing evidence.
4. Perform additional work where necessary.
5. Verify the revised result.

This does not mean every result must pass through endless review. Systems should define appropriate retry limits and clear completion criteria.

The key idea: Verification turns quality checks into a deliberate part of the execution process instead of relying entirely on the agent's confidence in its own answer.

23.7 Putting the Components Together

Let's combine these ideas in a practical example.

Imagine you're building a Deep Agent to power a customer support system for an online learning platform.

Students may ask about courses, payment issues, enrollment, certificates, or technical problems.

Some requests are straightforward. Others require the system to consult several sources, investigate an issue, and coordinate multiple actions.

23.7.1 The Architecture

The system could contain the following components:

Component	Responsibility
Orchestrator	Understands the request, creates the plan, and coordinates the work
Knowledge Retrieval Agent	Searches approved course documentation and support articles
Account Lookup Tool or Agent	Retrieves authorized enrollment and account information
Analysis Component	Uses the collected information to determine the likely answer or next step
Verification Component	Checks the proposed response against the available evidence and applicable rules
Persistent Storage	Maintains task state, relevant notes, and execution records

Not every component needs to be a separate agent. A database lookup may be better implemented as an ordinary function, while a specialized agent may be useful for interpreting a complex collection of documents.

The architecture should reflect the actual requirements rather than turning every operation into an agent.

23.7.2 A Request in Action

Suppose a student asks:

> I completed a course, but I can't find my certificate. Can you help me?

The system might proceed as follows.

Step 1: Understand the request

The orchestrator identifies that the student needs help locating a course certificate.

Step 2: Plan the investigation

It determines that it may need to review the certificate instructions and check the student's enrollment or completion status.

Step 3: Retrieve relevant information

The knowledge retrieval component searches approved documentation for the certificate requirements and delivery process.

Step 4: Check authorized account information

If the system has permission to access the student's account, it retrieves the relevant course completion status.

Step 5: Evaluate the findings

The orchestrator determines whether the available information explains the problem.

If the student has not completed the required steps, the response might explain what remains. If the completion requirements are satisfied but the certificate is missing, the issue may require further investigation.

Step 6: Verify the response

The system checks that its answer is consistent with the documentation and retrieved account information.

Step 7: Respond or escalate

The agent provides an evidence-based response or routes the unresolved issue to a human support representative.

Throughout the process, the system can record task progress and preserve relevant information needed to resume the investigation.

23.7.3 Why This Is More Than a Chatbot

A basic chatbot might respond with general advice about downloading a certificate.

The Deep Agent architecture can coordinate several operations to investigate the student's actual situation, while using tools and documented policies to

guide its response.

However, the architecture does not guarantee a correct result. The quality of the outcome still depends on the retrieved information, tool reliability, permissions, instructions, and verification process.

23.8 When Should You Build a Deep Agent?

Deep Agent architectures are useful for some demanding tasks, but they are not the right choice for everything.

23.8.1 Use a Deep Agent When...

- The task involves many interdependent subtasks.
- The execution plan may need to change as new information is discovered.
- The system must preserve substantial intermediate work.
- Different subtasks benefit from specialized capabilities.
- The final result requires independent verification.
- The task may need to recover from failures and resume unfinished work.

Examples include deep research, complex coding projects, extensive data analysis, and support investigations that involve multiple information sources.

23.8.2 A Simpler Agent May Be Better When...

- The task has only a few steps.
- The required tools and execution path are straightforward.
- Most operations can be completed within a short interaction.
- Additional planning and delegation would add unnecessary overhead.
- The task does not require substantial persistent memory.

For example, an agent that retrieves a shipping status and explains it probably does not need a planner, several sub-agents, an external memory system, and a dedicated verification agent.

A direct tool call with appropriate validation may be sufficient.

23.8.3 Consider the Trade-Offs

Deep Agent architectures can improve organization and support more complex work, but they also introduce costs and failure modes.

Benefit	Potential Trade-Off
Structured planning	Maintaining and updating the plan introduces additional work
Specialized agents	Delegation and coordination increase complexity
Persistent memory	Storage, retrieval, and stale information require management
Richer context	More context can increase latency and token costs
Dedicated verification	Additional checks require time and resources
Flexible execution	More decisions create more opportunities for unexpected behavior

The goal is to choose an architecture that matches the task's complexity and reliability requirements.

Start simple and introduce additional components only when they solve a demonstrated problem.

23.9 Try It Yourself

Task: Imagine you're building an AI system that investigates why a small business's sales dropped during the previous month.

The system has access to sales records, product information, customer feedback, and external market research.

Your first version is a simple agent that receives the question, performs a few searches, and generates a report.

However, the agent sometimes overlooks relevant data, repeats its searches, and makes unsupported claims about why sales declined.

Design a Deep Agent architecture to improve the system.

Answer the following questions:

1. How would the agent break down the investigation into tasks?
2. Which responsibilities should belong to the orchestrator, and which could be delegated to specialized components?
3. What information should be stored in persistent memory?
4. How should the system retrieve information when it needs to revisit a previous finding?
5. What should the verification component check before the final report is submitted?
6. How would the system handle missing data or conflicting evidence?

23.9.1 Example Answer

1. Create a structured research plan

The orchestrator could create tasks to examine changes in total revenue, product-level sales, customer feedback, and relevant market conditions.

It would record each task's status and update the plan as new findings reveal additional questions.

2. Separate responsibilities

The orchestrator would coordinate the investigation and integrate the results.

Specialized components could retrieve sales data, analyze product performance, summarize customer feedback, and research external market conditions.

A separate verification component could check important claims before the report is finalized.

3. Store useful information

Persistent storage could contain the research plan, task statuses, relevant data summaries, source references, calculations, and unresolved questions.

The system should distinguish verified findings from assumptions and hypotheses.

4. Retrieve previous findings

The agent could use structured queries to retrieve task statuses and sales metrics, then use keyword or semantic search to locate relevant research notes.

This allows it to revisit useful findings without repeating the entire investigation.

5. Verify the final report

The verification component should check whether calculations match the underlying data, important claims have supporting evidence, alternative explanations have been considered, and the report clearly distinguishes facts from hypotheses.

6. Handle uncertainty

If data is missing, the system should identify the limitation and investigate alternative sources when appropriate.

If evidence conflicts, it should compare the reliability and relevance of the sources instead of selecting whichever explanation appears most convincing.

If the available evidence does not establish why sales declined, the final report should present plausible explanations as hypotheses rather than confirmed causes.

23.10 What's Next

You've now learned how Deep Agents combine structured planning, orchestration, specialized sub-agents, persistent memory, context engineering, and verification to manage complex tasks.

The key takeaway is that **a Deep Agent is not simply an agent that performs more tool calls; it is a system designed to organize, preserve, coordinate, and verify work across a long-running task**.

These capabilities can make sophisticated AI applications more manageable, but they also require deliberate architectural choices and careful evaluation.

As you continue building agentic systems, pay particular attention to how you verify intermediate results and final outputs. Reliable execution is not just about completing a task—it's about establishing that the result satisfies the requirements and is supported by appropriate evidence.